Appunti S.I. da 3 CFU

Ionut Georgian Zbirciog

4 febbraio 2026

Indice

1 Gruppi **2**
1.1 Il Gruppo $\mathbb{Z}_p^*$ . . . 2

2 Introduzione al Secret Sharing **4**

3 Shamir Secret Sharing **6**
3.1 Schema $(2, n)$ . . . 6
3.2 Estensione a (t, n) . . . 7
3.3 Secret Sharing $(2, 2)$ for RSA . . . 7

4 Secret Sharing for Secure Multiparty Computation **8**
4.1 TTP vs SMC . . . 9
4.2 Problemi di Implementazione . . . 9
4.3 Costruzione . . . 10

5 Verifiabile Secret Sharing **12**
5.1 Feldman Scheme . . . 13
5.2 Pedersen Scheme . . . 14
5.3 Distributed Key Generation . . . 16

6 Threshold and Policy-Based Cryptography **18**
6.1 Threshold Encryption: ElGamal . . . 18
6.2 Threshold Signatures: RSA . . . 19
6.2.1 RSA Classico . . . 19
6.2.2 Threshold RSA . . . 19

7 Linear Secret Sharing & Access Control Matrix **21**
7.1 LSSS e MSP . . . 21
7.2 LSSS e Strutture di Accesso . . . 23

8 Elliptic Curve **26**
8.1 EC Group for Cryptography . . . 27
8.2 Algoritmio Double-and-Add . . . 27
8.3 ECDH . . . 28
8.4 ECDSA . . . 29

9 Bilinear Maps **31**
9.1 Decisional Diffie-Hellman . . . 31
9.2 Joux's 3-Party Diffie-Hellman . . . 33
9.3 Boneh and Franklin's IBE Scheme . . . 33

1 Gruppi

Definizione. Un gruppo $(\mathbb{G}, \star)$ è una struttura algebrica composta da un insieme di elementi $\mathbb{G}$ e un'operazione $\star$. Per essere definito tale, deve soddisfare quattro proprietà fondamentali:

Chiusura: Per ogni coppia di elementi $a,\ b \in \mathbb{G}$, il risultato $a \star b \in \mathbb{G}$. → $\forall a,b \in G \Rightarrow$ risultato op $a \star b \in G$

Identità (Elemento Neutro): Esiste un elemento I, tale che, per ogni $a \in \mathbb{G}$, si ha che $a \star I = I \star a = a$.

Inverso: Per ogni elemento $a \in \mathbb{G}$, esiste un elemento $a^{-1} \in \mathbb{G}$ tale che $a \star a^{-1} = I$.

Associatività: Per ogni $a,\ b,\ c \in \mathbb{G}$, vale che $(a \star b) \star c = a \star (b \star c)$.

Inoltre, diremo che il gruppo è **Abeliano** se vale anche la proprietà di **commutativa**, ossia $a \star b = b \star a$.

1.1 Il Gruppo $\mathbb{Z}_p^*$

Il gruppo $\mathbb{Z}_p^* = (\mathbb{Z}_p, \cdot)$ è il gruppo moltiplicativo modulo un numero primo p. È un gruppo finito composto da $p-1$ elementi:

$$\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$$

L'unica operazione considerata è **moltiplicazione modulo** p; la somma viene esclusa, altrimenti si parlerebbe di un campo $\mathbb{F}_p$.

In generale un elemento x ha un inverso modulo N se e solo se $gcd(x, N) = 1$. Poiché p è primo, tutti gli elementi da 1 a $p-1$ hanno un inverso. Per gruppi molto grandi, l'inverso si calcola con l'**Algoritmo Euclideo Esteso**.

Lo zero non fa parte del gruppo perché non ammette inverso moltiplicativo.

Esempio. Consideriamo il gruppo $\mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Gli inversi degli elementi sono:

$1 \to 1$;
$2 \to 6$; infatti $2 \cdot 6 \mod 11 = 12 \mod 11 = 1$.
$3 \to 4$; infatti $3 \cdot 4 \mod 11 = 12 \mod 11 = 1$.
$\vdots$

Definizione (Generatore). Sia $(\mathbb{G}, \star)$ un gruppo finito di cardinalità m. Un elemento $g \in \mathbb{G}$ è un **generatore** se il sottogruppo generato dalle sue potenze coincide con tutto il gruppo:

$$\langle g \rangle = \{g^0, g^1, \ldots, g^{m-1}\} = \mathbb{G}.$$

Equivalentemente, l'ordine di g è m, cioè il minimo $r \geq 1$ tale che $g^r = I$ soddisfa $r = m$. Se m è primo, allora ogni elemento diverso dall'identità è un generatore e il gruppo è detto **gruppo di ordine primo**.

Consideriamo il gruppo $\mathbb{Z}_p^*$. Il suo ordine è $|\mathbb{Z}_p^*| = p-1$. Poiché p è primo, per $p > 2$ vale che $p-1$ è pari; quindi, in generale l'ordine non è primo (eccezione: $p = 3 \Rightarrow p-1 = 2$). Inoltre, $\mathbb{Z}_p^*$ è ciclico: esiste $g \in \mathbb{Z}_p^*$ tale che $\langle g \rangle = \mathbb{Z}_p^*$.

\# GENERATORI IN $\mathbb{Z}_p^* = \Phi(p-1)$
ES $\mathbb{Z}_{11}^* \Rightarrow$ \# GENERATORI $= \Phi(11-1) = \Phi(10) = (5-1)(2-1) = 4$ ✓

Esempio. Consideriamo $\mathbb{Z}_{11}^*$, e analizziamo i suoi generatori.

g	$\{g^1, g^2, g^3, \ldots, g^{10}\}$	Generatore	Ordine Sottogruppo
2	{2, 4, 8, 5, 10, 9, 7, 3, 6, 1}	SI	10
3	{3, 9, 5, 4, 1, 3, 9, 5, 4, 1}	NO	5
4	{4, 5, 9, 3, 1, 4, 5, 9, 3, 1}	NO	5
5	{5, 3, 4, 9, 1, 5, 3, 4, 9, 1}	NO	5
6	{6, 3, 7, 9, 10, 5, 8, 4, 2, 1}	SI	10
7	{7, 5, 2, 3, 10, 4, 6, 9, 8, 1}	SI	10
8	{8, 9, 6, 4, 10, 3, 2, 5, 7, 1}	SI	10
9	{9, 4, 3, 5, 1, 9, 4, 3, 5, 1}	NO	5
10	{10, 1, 10, 1, 10, 1, 10, 1, 10, 1}	NO	2

Tabella 1: Analisi dei generatori e sottogruppi di $\mathbb{Z}_{11}^*$

Definizione (Primo forte). Un numero primo p si dice **primo forte** se $p = 2q + 1$ con q primo. Nel gruppo moltiplicativo $\mathbb{Z}_p^*$ l'ordine è $p - 1 = 2q$; quindi gli ordini possibili degli elementi sono 1 (l'identità), 2 (solo $p-1$), q oppure $2q$. In particolare, esclusi 1 e $p-1$, ogni elemento è o un generatore dell'intero gruppo (ordine $2q$) oppure genera il sottogruppo ciclico di ordine primo q.

Scegliere p e q entrambi grandi riduce il rischio di sottogruppi di piccolo ordine, problema tipico quando $p - 1$ ha molti piccoli fattori primi.

Definizione (Residuo Quadratico). Sia p primo e si lavori in $\mathbb{Z}_p^*$. Un elemento $x \in \mathbb{Z}_p^*$ è un **residuo quadratico** se esiste $a \in \mathbb{Z}_p^*$ tale che $a^2 \equiv x \pmod p$. L'insieme

$$QR_p = \{a^2 \bmod p : a \in \mathbb{Z}_p^*\}$$

è un sottogruppo di $\mathbb{Z}_p^*$ di ordine $\frac{p-1}{2}$, poiché l'applicazione $a \mapsto a^2$ è 2-a-1 (infatti a e $-a$[1] hanno lo stesso quadrato).

Criterio di Eulero e simbolo di Legendre. Per $a \in \mathbb{Z}_p^*$ vale

$$a \in QR_p \iff a^{\frac{p-1}{2}} \equiv 1 \pmod p,$$

e il simbolo di Legendre è

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{se } p \mid a, \\ 1 & \text{se } a \in QR_p, \\ -1 & \text{altrimenti.} \end{cases}$$

Se p è *primo forte*, $p = 2q + 1$ con q primo, allora $|QR_p| = \frac{p-1}{2} = q$ è primo.

ESERCIZIO:
DETERMINARE SE 3 È GEN DI $\mathbb{Z}_{23}$
SOL
ESSENDO 23 PRIMO ⇒ ORDINE DI $\mathbb{Z}_{23}$ = 23-1 = 22
PER VERIFICARE CHE 3 È GEN DI $\mathbb{Z}_{23}$ BISOGNA VERIFICARE CHE $3^d \not\equiv 1 \pmod{23}$ ∀ DIVISORE PROPRIO MASSIMALE DELL'ORDINE DEL GRUPPO (22)
I FATT. PRIMI DI 22 SONO 11 E 2, QUINDI PROVIAMO
$3^2 \equiv 9 \pmod{23}$, $3^{11} \equiv 1 \pmod{23}$ ⇒ L'ORDINE DI 3 È 11 ⇒ 3 NON È GENERATORE

[1] $(p-a)^2 \mod p = p^2 + a^2 - 2pa \mod p = a^2$

2 Introduzione al Secret Sharing

Il concetto fondamentale del **Secret Sharing** è dividere un segreto S in vari pezzi (chiamati *shares* o *quote*) e distribuirli a un gruppo di partecipanti (n persone). L'obiettivo è che il segreto possa essere ricostruito solo combinando un certo numero di queste quote, mantenendo la sicurezza totale se il numero di quote è insufficiente.

Consideriamo innanzitutto un approccio ingenuo al secret sharing. La domanda è: **come condividere un segreto tra due persone senza rivelarlo a nessuna delle due?** Sia $S \in \{0,1\}^k$ il segreto da condividere tra A e B.

Un'idea semplice è consegnare metà del segreto ad A e l'altra metà a B. Questo però è sbagliato: la probabilità di indovinare il segreto prima della suddivisione è

$$\Pr[\text{indovinare } S] = \frac{1}{2^k},$$

mentre dopo aver visto una delle due metà diventa

$$\Pr[\text{indovinare } S] = \frac{1}{2^{k/2}},$$

riducendo sensibilmente la sicurezza.

Una strategia più sicura è la seguente:

- Sia $S \in \{0,1\}^k$ il segreto.
- Sia $R \in_{u.a.r.} \{0,1\}^k$ una stringa scelta uniformemente a caso.
- Definiamo $SXR := S \oplus R$, ossia lo XOR tra S e R.
- Si consegna R ad A e SXR a B.
- Per ricostruire S è sufficiente calcolare

$$S = SXR \oplus R$$

Questo schema garantisce la *perfect secrecy*: la quota assegnata ad A è la stringa casuale R, mentre quella per B è $S \oplus R$; ciascuna, presa singolarmente, è uniformemente distribuita e indipendente da S. Ne segue che osservare una sola quota non rivela alcuna informazione sul segreto.

In sintesi, questo schema, sebbene semplice, è più sicuro del precedente e garantisce *perfect secrecy* (One-Time Pad): ogni singola quota, presa da sola, è uniformemente distribuita e indipendente da S. Di conseguenza, osservare una sola quota non riduce l'incertezza sul segreto; la probabilità di indovinare S resta $\Pr[\text{indovinare } S] = 2^{-k}$.

Esempio. Sia $S = 0010.1101$ e sia $R = 1011.0100$. $SXR = S \oplus R = 1001.1001$.

- A riceve $R = 1011.0100$.
- B riceve $SXR = 1001.1001$.
- Per ricostruire S è necessario conoscere sia R che SXR, quindi

$$S = R \oplus SXR = 0010.1101$$

→ ESEMPIO (2,2)
→ SCHEMA (n,n) TRIVIA = XOR DI TUTTE LE SHARES

Per usare questo schema, invece di considerare lo XOR, possiamo usare la somma modulare, ossia:

- Sia $S \in \mathbb{Z}_k$ il segreto.
- Sia $R \in_{u.a.r.} \mathbb{Z}_k$ una stringa scelta uniformemente a caso.
- Definiamo $SXR := S - R \mod 256$.

- Si consegna R ad A e SXR a B.
- Per ricostruire S è sufficiente calcolare

$$S = SXR + R \mod 256$$

Esempio. Consideriamo $\mathbb{Z}_{256}$ e siano $S = 45$ e $R = 180$. $SXR = S - R = -135 \mod 256 = 121$.

- A riceve $R = 180$.
- B riceve $SXR = 121$.
- Per ricostruire S è necessario conoscere sia R che SXR, quindi

$$S = R + SXR \mod 256 = 180 + 121 \mod 256 = 45$$

Infine, lo schema si generalizza a n partecipanti, ottenendo uno schema (n, n) in cui tutte le n quote sono necessarie per ricostruire il segreto; qualsiasi insieme di al più $n - 1$ quote non rivela alcuna informazione (*perfect secrecy*).

SCHEMA SENZA ARITMETICA MODULARE = SCHEMA IMPERFETTO
→ VA USATA L'ARITMETICA MODULARE, CON CONDIZIONE CHE P > DOMINIO DEL SEGRETO
ES: SE SEGRETO S ∈ (0,100) ⇒ P = 101 VA BENISSIMO

3 Shamir Secret Sharing

Come introdotto, lo schema precedente è di tipo (n, n). Vogliamo ora passare a uno schema (t, n), con $1 \leq t \leq n$: dati n partecipanti, è sufficiente che almeno t combinino le proprie quote per ricostruire il segreto, mentre qualunque insieme di meno di t quote non rivela alcuna informazione su S.

3.1 Schema $(2, n)$

Consideriamo lo spazio dei polinomi di grado 1, ossia l'insieme dei polinomi che descrivono le rette. Per identificare univocamente una retta è necessario conoscere 2 punti appartenti a essa.

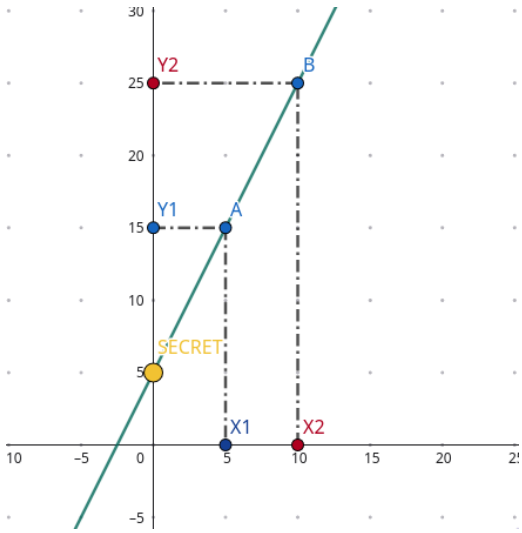


Figura 1: Spiegazione grafica dello schema $(2, n)$

Dalla Figura 1, il segreto coincide con $y(0)$, ossia il punto $(0, S)$.

Lo schema procede così:

- Il dealer sceglie u.a.r. un coefficiente a e definisce la retta $y(x) = S + a\,x$.
- Assegna a ciascun partecipante un punto sulla retta: il partecipante i riceve $P_i = (x_i, y(x_i))$ (con x_i anche scelto a caso).
- Per ricostruire S, è sufficiente che due partecipanti forniscano i punti $P_i = (x_i, y_i)$ e $P_j = (x_j, y_j)$; interpolando la retta si ottiene

$$S = y(0) = y_j + \frac{-x_j}{x_i - x_j}(y_i - y_j)$$

$= y_j \frac{-x_j}{x_j - x_i} + y_i \frac{-x_i}{x_i - x_j}$

Retta passante per due punti

Dati due punti $P_i = (x_i, y_i)$ e $P_j = (x_j, y_j)$, l'unica retta passante per questi punto è data da:

$$\frac{y - y_j}{y_i - y_j} = \frac{x - x_j}{x_i - x_j} \rightarrow y = y_j + \frac{x - x_j}{x_i - x_j}(y_i - y_j)$$

Esempio. Siano $P_i = (3, 84)$ e $P_j = (1, 54)$, allora

$$S = y_j + \frac{-x_j}{x_i - x_j}(y_i - y_j) = 54\frac{3}{2} - 84\frac{1}{2} = 39$$

$P_i = (3, 84)$; $P_j = (1, 54)$
x_i, y_i $\quad$ x_j, y_j

$S = 54 + \frac{-1}{3-1}(84 - 54) =$

$= 54 - \frac{1}{2}(84 - 54) = 54 - 42 + 28 = 39$

• p > m, w p > m²x

3.2 Estensione a (t, n)

L'idea è codificare S come termine noto di un polinomio di grado $t-1$ definito su un campo finito $\mathbb{F}_q$ (tipicamente $\mathbb{Z}_p$ con p primo e $p > n$[2]). In questo modo, qualunque insieme di t punti determina univocamente il polinomio (e quindi S), mentre con meno di t punti il segreto rimane perfettamente nascosto.

1. Scelta dei parametri: si fissa un campo $\mathbb{F}_q$ con $q > n$.
2. Generazione: il dealer estrae $a_1, \dots, a_{t-1} \leftarrow_{u.a.r.} \mathbb{F}_q$ e definisce
$$f(x) = S + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \in \mathbb{F}_q[x].$$
3. Distribuzione: si scelgono $x_1, \dots, x_n \in \mathbb{F}_q^*$ distinti e si consegna a ogni partecipante i la quota
$$P_i = (x_i,\ y_i = f(x_i)).$$
4. Ricostruzione: Il dealer riceve un'qualsiasi sottoinsieme di t segreti, e ricostruisce S interpolando il polinomio usando l'interpolazione di Lagrange: Dato un insieme di t quote (x_i, y_i) su f (grado $t-1$), il segreto vale
$$S = f(0) = \sum_{i=1}^{t} y_i \lambda_i \qquad \text{dove} \qquad \lambda_i = \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{-x_j}{x_i - x_j} \in \mathbb{F}_q.$$

λ_i = i-esimo polinomio di Lagrange
↳ qui può variare da 1 a t ma su tutto n

Proprietà.

Correttezza: t punti determinano univocamente f, quindi recuperano $S = f(0)$.

Perfect Secrecy: con meno di t punti, per ogni $S' \in \mathbb{F}_q$ esiste un polinomio di grado $t-1$ coerente con le quote osservate e $f(0) = S'$, dunque le quote non rivelano informazione su S.

Esempio (Schema $(3,4)$). Sia $\mathbb{F}_q = \mathbb{Z}_{101}$, consideriamo il polinomio
$$p(x) = 32 + 52x + 3x^2 \mod 101.$$
I punti condivisi sono: $P_1 = (1, 87)$, $P_2 = (2, 47)$, $P_3 = (3, 13)$, $P_4 = (4, 86)$. Calcoliamo S usando P_1, P_2, P_3.
$$S = p(0) = \sum_{i=1}^{3} y_i \lambda_i = 87 \cdot 3 + 47 \cdot (-3) + 13 \cdot 1 \mod 101 = 32.$$

$\lambda_1 = \prod_{\substack{j=1 \\ j \neq 1}}^{3} \frac{-x_j}{x_1 - x_j} = \frac{-2}{1-2} \cdot \frac{-3}{1-3} = 2 \cdot \frac{3}{2} = 3$

3.3 Secret Sharing $(2,2)$ for RSA

l'obiettivo è suddividere l'esponente segreto d in due quote, così che nessuna parte possa operare da sola; firma e decifratura si ottengono solo cooperando, senza mai ricostruire d per intero.

Sia $N = pq$ con $\phi(N) = (p-1)(q-1)$ e $e \cdot d \equiv 1 \pmod{\phi(N)}$.

Setup (divisione del segreto). Si sceglie $d_1 \leftarrow_{u.a.r.} \mathbb{Z}_{\phi(N)}$ e si pone $d_2 \equiv d - d_1 \pmod{\phi(N)}$, in modo che $d_1 + d_2 \equiv d \pmod{\phi(N)}$.

Firma/Decifratura. La Parte 1 calcola $\sigma_1 \equiv H(m)^{d_1} \pmod N$, la Parte 2 calcola $\sigma_2 \equiv H(m)^{d_2} \pmod N$, e l'aggregatore ottiene
$$\sigma \equiv \sigma_1 \cdot \sigma_2 \equiv H(m)^{d_1} \cdot H(m)^{d_2} \equiv H(m)^{d} \pmod N.$$
La verifica è quella standard RSA: $\sigma^e \equiv H(m) \pmod N$. In modo analogo per la decifratura: $c^{d_1} \cdot c^{d_2} \equiv c^d \pmod N$.

[2] Se $n = 100$, $p = 101$ è un primo valido.

4 Secret Sharing for Secure Multiparty Computation

Definizione (SMC). SMC sta per **Secure Multiparty Computation** è consiste nel calcolare il risultato di una funzione che prende in input dati da più persone, *senza che nessuno debba rivelare il proprio dato agli altri.*

Sia $\mathcal{P} = \{P_1, P_2, \ldots, P_n\}$ un insieme di n parti distinte. Ogni parte P_i detiene un valroe di input privato denotato con z_i. L'obiettivo del protocollo è calcolare una funzione deterministica f sugli input di tutte le parti:

$$y = f(z_1, z_2, \ldots, z_n)$$

soggetta ai seguenti vingoli:

Correttezza e Pubblicazione: Il risultato y viene calcolato correttamente ed è reso pubblico.

Privacy degli Input: Nessuna informazione riguradante i valori privati z_i viene rivelata durante il processo di calcolo, ad eccezione di quanto può essere logicamente dedotto dal risultato finale y stesso.

A noi ci interessa nello specifico il caso dell'aggregazione lineare (addizione pesata), ossia

$$f(z_1, z_2, \ldots, z_n) = \sum_{i=1}^{n} \alpha_i z_i = \alpha_1 z_1 + \alpha_2 z_2 + \cdots + \alpha_n z_n$$

dove α_i, rappresentano dei coefficienti o pesi.

Se la funzione f appartiene a questa classe, la SMC può essere implementata in modo molto efficiente utilizzando scehmi di *secret sharing.*

Prima di proseguire, è necessario introdurre la **proprietà omomorfica**, principio fondamentale alla base della SMC.

Definizione (Proprietà Omomorfica). Siano dati due segreti distinti, S_F e S_G. Nello schema di Shamir, questi segreti sono nascosti nei temini noti di due polinomi $f(x)$ e $g(x)$ di grado $t-1$, tali che:

$$f(0) = S_F \quad \text{e} \quad g(0) = S_G$$

Ogni partecipanti i (con $i = 1, \ldots, n$) possiede una **share** (quota) per ciascun segreto, calcolato valutando i polinomi nel punto i.

$$y_{i,F} = f(i) \qquad \text{e} \qquad y_{i,G} = g(i)$$

La proprietà omomorfica afferma che **la somma delle share è uguale alla share della somma**.

$$y_{i,\text{somma}} = y_{i,F} + y_{i,G} = f(i) + g(i) = (f+g)(i)$$

Cioè, il valore $y_{i,\text{somma}}$ corrisponde esattamente alla valutazione nel punto i del polinomio somma $(f+g)(i)$. Quindi, poiché l'operazione di somma tra polinomi preserva il termine noto (la somma dei termini noti è il termine noto del polinomio somma), il segreto ricostruibile dalle nuove quote aggregate sarà:

$$S_{\text{totale}} = (f+g)(0) = f(0) + g(0) = S_F + S_G$$

Questo dimostra che rivelare le quote composite $y_{i,\text{somma}}$ permette di ricostruire la **somma dei segreti** $(S_F + S_G)$, **ma non rivela alcuna informazione** sui singoli segreti originali S_F e S_G.

Esempio. Consideriamo il seguente esempio, siano:

- **Modulo:** $p = 11$ (campo finito $\mathbb{Z}_{11}$)
- **Soglia:** $t = 2$ (bastano 2 quote per ricostruire)
- **Segreti scelti:** $S_F = 3$, $S_G = 4$

Il Dealer genera due polinomi casuali di grado $t - 1 = 1$:

$$\begin{cases} f(x) = 3 + 2x \pmod{11} & (\text{nasconde } S_F) \\ g(x) = 4 + 5x \pmod{11} & (\text{nasconde } S_G) \end{cases}$$

Ogni partecipante P_i riceve le quote $f(i)$ e $g(i)$ e calcola localmente la loro somma.

Utente (x)	**Quota Segreto F** $(y_{i,F})$	**Quota Segreto G** $(y_{i,G})$	**Calcolo Locale** $(f+g)$	**Quota Somma** $(y_{i,\text{sum}})$
P_1 $(x=1)$	$3+2(1) = \mathbf{5}$	$4+5(1) = \mathbf{9}$	$5+9 = 14$	**3** (mod 11)
P_2 $(x=2)$	$3+2(2) = \mathbf{7}$	$4+5(2) = 14 \equiv \mathbf{3}$	$7+3 = 10$	**10** (mod 11)
P_3 $(x=3)$	$3+2(3) = \mathbf{9}$	$4+5(3) = 19 \equiv \mathbf{8}$	$9+8 = 17$	**6** (mod 11)

Tabella 2: Calcolo delle quote aggregate

Le nuove quote calcolate sono i punti: $(1, 3)$, $(2, 10)$, $(3, 6)$. Questi punti appartengono al **polinomio somma** teorico:

$$h(x) = f(x) + g(x) = (3 + 2x) + (4 + 5x) = \mathbf{7} + \mathbf{7x} \pmod{11}$$

Il termine noto di $h(x)$ è **7**, che corrisponde esattamente alla somma dei segreti originali $(3 + 4)$.

Verifica della consistenza per P_2:

$$y = 7 + 7(2) = 7 + 14 = 21 \equiv \mathbf{10} \pmod{11} \quad \text{(Corretto!)}$$

Inoltre, presi due punti, $(1, 3)$ e $(2, 10)$, la retta passante per questi punti è:

$$y = 10 + \frac{x-2}{1-2}(3 - 10) = 10 + 7x - 14 \mod 11 = 7x - 4 \mod 11 = 7x + 7$$

4.1 TTP vs SMC

Tradizionalmente, per eseguire un'aggregazione, tutti gli utenti inviano i dati in chiaro a un server centrale, una **Trusted Third Party (TTP)**. Per calcolare, ad esempio, una somma, tale ente deve accedere a tutti i dati privati. Se la TTP viene compromessa o si comporta in modo malevolo, la privacy di tutti è a rischio. Questo modello è semplice e funziona, ma non offre garanzie di riservatezza nei confronti di chi esegue il calcolo.

In alternativa, si adotta un insieme di server distribuiti, detti *Privacy Peers*, che operano su dati condivisi tramite secret sharing. L'utente P_1, anziché inviare z_1 in chiaro, lo suddivide in quote e ne invia una a ciascun peer. Ogni peer riceve soltanto frammenti privi di significato presi singolarmente. Ciascun peer somma localmente le quote ricevute, ottenendo una quota del risultato finale $Share(\text{RES})$, senza mai vedere i dati originali; il risultato si ricostruisce solo combinando un numero sufficiente di quote.

4.2 Problemi di Implementazione

Per utilizzare il sistema in sicurezza è opportuno imporre alcuni vincoli operativi.

Input peers (n): almeno 3. Con soli 2 partecipanti, A e B, se il risultato aggregato (es. la somma) è pubblico, A può sottrarre il proprio contributo e dedurre esattamente quello di B. Con 3 o più partecipanti questo non è più possibile.

Privacy peers (k): almeno 2. Un solo privacy peer riceverebbe abbastanza quote da ricostruire i segreti. Aumentare il numero di privacy peers riduce il rischio di collusione.

Chi sono i privacy peers. Gli stessi input peers possono fungere da privacy peers: non serve acquistare server esterni; gli utenti possono scambiarsi le quote tra loro ed eseguire il calcolo distribuito.

4.3 Costruzione

Basato sullo schema di Shamir, il protocollo permette di calcolare la somma degli input senza rivelarli. Il processo si articola in tre fasi distinte.

1. **Input Peer i (Fase di invio)** L'utente i possiede un dato privato z_i (es. il proprio salario). L'utente genera un polinomio casuale $p_i(x)$ di grado $t-1$, imponendo che il termine noto sia il proprio segreto:

$$p_i(0) = z_i$$

A questo punto, l'utente valuta il polinomio nei punti corrispondenti agli indici dei server e invia le quote in modo privato, non dovendo comunicare tra loro né scambiarsi chiavi.

- Invia $p_i(1)$ al Privacy Peer 1
- Invia $p_i(2)$ al Privacy Peer 2

...

- Invia $p_i(k)$ al Privacy Peer k

2. **Privacy Peer m (Fase di calcolo)** Ogni server di calcolo riceve una quota da ciascun utente, $p_1(m), p_2(m), \ldots, p_n(m)$. Calcola la somma aritmetica delle quote ricevute. Grazie alla proprietà omomorfica, questa somma corrisponde a un punto del "polinomio somma":

$$\text{RES}(m) = \sum_{j=1}^{n} p_j(m) = p_1(m) + p_2(m) + \cdots + p_n(m)$$

Il server pubblica il risultato parziale aggregato RES(m). Questo valore, da solo, non rivela nulla sui dati originali.

3. **Public (Fase di ricostruzione)** Questa fase può essere eseguita da chiunque abbia accesso ai risultati pubblicati. Si raccolgono almeno t valori pubblici RES(m) dai vari Privacy Peers. Utilizzando l'interpolazione di Lagrange, si ricostruisce il termine noto del polinomio risultante, che corrisponde esattamente alla somma totale degli input:

$$Z_{\text{totale}} = \sum_{i=1}^{n} z_i$$

Se si sceglie di usare tutti i privacy peer per la ricostruzione, allora si può usare l'approccio banale di secret sharing.

4D APPROCCIO SCHEMA (n, n) CON MODULO

• **Esempio.** Consideriamo un esempio basato su uno schema di secret sharing $(3,3)$ in cui i privacy peer coincidono con gli input peer. Trattandosi di uno schema (k,k), è sufficiente adottare l'approccio banale.

Consideriamo 3 parti (P_1, P_2, P_3) che vogliono calcolare la somma dei loro valori segreti $(14, 26, 38)$ utilizzando l'aritmetica **modulo 100**.

	Quote inviate a...			
Mittente (Z_i)	**Party 1**	**Party 2**	**Party 3**	**Verifica**
Party 1 (14)	**34**	89	91	$(34+89+91 \equiv 14)$
Party 2 (26)	66	**11**	49	$(66+11+49 \equiv 26)$
Party 3 (38)	47	51	**40**	$(47+51+40 \equiv 38)$
SOMMA LOCALE (Risultato)	↓ **47**	↓ **51**	↓ **80**	

Tabella 3: Matrice di scambio delle quote (Modulo 100). Sulla diagonale in grassetto le quote trattenute.

Si sommano i risultati parziali pubblicati dai tre nodi:

$$\text{Totale} = R_1 + R_2 + R_3 = 47 + 51 + 80 = 178 \equiv \mathbf{78} \pmod{100}$$

Il risultato corrisponde esattamente alla somma degli input segreti $(14 + 26 + 38 = 78)$.

• SCHEMA $(3,3)$ SU $\mathbb{Z}_{100}$ → SERVONO TUTTE LE PARTI PER RICOSTRUIRE IL SEGRETO

P_1 + SEGRETO $z_1 = 14$
P_2 ⇒ —— $z_2 = 26$
P_3 ⇒ —— $z_3 = 38$
→ OBIETTIVO È CALCOLARE $z_1+z_2+z_3 \pmod{100}$
⇒ $14+26+38 \pmod{100} = 78 \pmod{100}$

FASE 1 : SS ADDITIVO

OGNI PARTE SPEZZA IL SEGRETO in 3 QUOTE CASUALI LA CUI SOMMA MOD 100 fa IL SEGRETO z_i:

P_1 (14) → SCEGLIE U.A.R 34, 89, 91 → $34+89+91 \pmod{100} \equiv 14 \pmod{100}$ ✓
↳ P_1 invia 34 a P_1, 89 a P_2 e 91 a P_3

P_2 (26) → U.A.R 66, 11, 49
↳ P_2 invia 66 a P_1, 11 a P_2 e 49 a P_3

P_3 (38) → U.A.R 47, 51, 40
↳ P_3 invia 47 a P_1, 51 a P_2 e 40 a P_3

QUINDI:
- P_1 RICEVE 34, 66 e 47 → CALCOLA SOMMA LOCALE : $R_1 = 34+66+47 \pmod{100} \equiv 47 \pmod{100}$
- P_2 —— 89, 11 e 51 → —— $R_2 = 89+11+51 \pmod{100} \equiv 51 \pmod{100}$
- P_3 —— 91, 49 e 40 → —— $R_3 = 91+49+40 \pmod{100} \equiv 80 \pmod{100}$

ORA TUTTI PUBBLICANO R_1, R_2 E R_3; IL SEGRETO VIENE QUINDI RICAVATO COME
$S = R_1 + R_2 + R_3 \pmod{100} \Rightarrow 47+51+80 \pmod{100} \equiv 78 \pmod{100}$ ✓

5 Verifiabile Secret Sharing

Nei modelli di *secret sharing* tradizionali, si assume che tutti i partecipanti (sia il Dealer che distribuisce i pezzi del segreto, sia le Parti che li conservano) siano onesti. Tuttavia, questo modello presenta dei limiti critici:

Dealer Malvagio: Non c'è modo di verificare se il Dealer stia distribuendo quote coerenti tra loro.

Parti Corrotte: Non si può rilevare se un partecipante riveli una quota falsa al momento della ricostruzione del segreto.

La VSS introduce la possibilità per ogni partecipante di verificare matematicamente la validità delle quote ricevute senza dover conoscere il segreto intero. Questo avviene solitamente attraverso i Commitments.

In questo capitolo verranno discurssi 3 schemi principali:

Schema di Feldman (1987): Il primo schema non interattivo basato sulla difficoltà del logaritmo discreto. È efficiente ma solo computazionalmente sicuro, poiché il commitment del segreto (g^s) potrebbe trapelare informazioni se il segreto è molto piccolo.

Schema di Pedersen (1991): Risolve il problema della sicurezza di Feldman utilizzando commitments perfettamente sicuri (**hiding**), garantendo che nessuna informazione sul segreto venga trapelata, a patto di una sicurezza computazionale nel vincolo (**binding**).

Distributed Key Generation (DKG): Una tecnica avanzata che permette a un gruppo di generare una coppia di chiavi (pubblica/privata) in modo che la chiave pubblica sia nota a tutti, ma nessuno conosca la chiave privata completa finché non è necessario.

Definizione (Commitment). Un **commitment (impegno)** è un protocollo che permette a un utente di *sigillare* un valore segreto, rendendolo pubblico in una forma indecifrabile per poi rivelarlo in un secondo momento.

Per essere considerato valido, un commitment deve soddisfare due proprità fondamentali:

Hiding (Segretezza): Il ricevitore non deve essere in grado di estrarre alcuna informazione sul valore originale guardando solo l'impegno pubblico.

Binding (Vincolo): Una volta inviato l'impegno, il mittente non può più cambiare idea o modificare il valore originale; può solo rivelare quello che ha inizialmente *sigillato*.

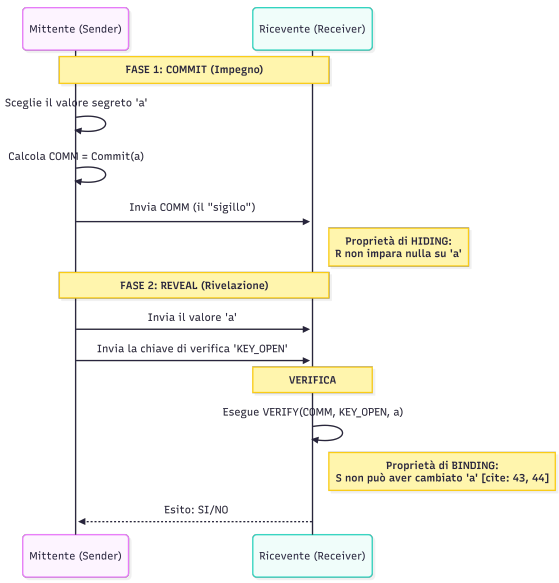


Figura 2: Processo di Commitment

5.1 Feldman Scheme

Lo schema di Feldman, proposto nel 1987, è stato il primo schema di Verifiable Secret Sharing (VSS) di tipo non interattivo. Si basa sull'estensione del classico schema di Shamir, aggiungendo una fase di verifica basata sulla difficoltà del logaritmo discreto.

Si articola nelle due figure chiave: il **Dealer** (chi distribuisce) e il **Verifier** (chi riceve).

Dealer: Come nello schema classico di Shamir, il Dealer crea un polinomio $p(x)$ di grato $t-1$, dove il termine noto è il segreto S:

$$p(x) = S + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}$$

Invia poi a ogni partecipante i la propria quota privata $y_i = p(x_i)$. Infine, pubblica tutti i commitment calcolati come potenze di un generatore g modulo un numero primo p:

$$\begin{aligned} c_0 &= g^S \mod p \\ c_1 &= g^{a_1} \mod p \\ &\vdots \\ c_{t-1} &= g^{a_{t-1}} \mod p \end{aligned}$$

Verifier: Ogni partecipante i, ricevuta la sua quota (x_i, y_i), può verificare se il Dealer è stato onesto confrontando la quota con i commit pubblici. Il partecipante controlla se la seguente uguaglianza è verificata:

$$\begin{aligned} c_0 \cdot c_1^{x_i} \cdot c_2^{x_i^2} \cdots c_{t-1}^{x_i^{t-1}} &= (g^S) \cdot (g^{a_1})^{x_i} \cdot (g^{a_2})^{x_i^2} \cdots (g^{a_{t-1}})^{x_i^{t-1}} \\ &= g^S \cdot g^{a_1 x_i} \cdot g^{a_2 x_i^2} \cdots g^{a_{t-1} x_i^{t-1}} \\ &= g^{S + a_1 x_i + a_2 x_i^2 + \cdots + a_{t-1} x_i^{t-1}} \\ &= g^{p(x_i)} = g^{y_i} \end{aligned}$$

Se l'uguaglianza è verificata, allora il Dealer è onesto.

Inolte, durante la fase di ricostruzione del segreto, i partecipanti si **verificano reciprocamente** per individuare eventuali cheater. Quando il partecipante i rivela la propria quota y_i per la ricostruzione, gli altri confrontano il valore con i commitment pubblici (c_j) pubblicati dal Dealer all'inizio, verificando che la quota corrisponda a quanto impegnato. Questo consente di individuare e bloccare quote false durante la ricostruzione.

Purtroppo lo schema è solo computazionalmente sicuro (a differenza di Shamir, che è incondizionatamente sicuro) poiché si basa sulla difficoltà del logaritmo discreto. Inoltre, il valore pubblico $c_0 = g^S$ può trapelare informazione sul segreto: se S proviene da un dominio piccolo (ad es. $S \in [1, 1000]$), un avversario può eseguire un attacco a dizionario calcolando $g^1, g^2, \ldots$ fino a trovare c_0. Il commitment non è quindi perfettamente *hiding*.

Per rendere lo schema più robusto, è consigliato utilizzare numeri primi tali che $p = 2q + 1$ con p, q primi, e di operare all'interno di sottogruppi di ordine q per limitare le informazioni che trapelano.

COMMITMENT DI FELDMAN:
- (COMPUTATIONALLY) HIDING
- (PERFECT) BINDING

5.2 Pedersen Scheme

Lo schema di Pedersen (1991) nasce proprio per risolvere il limite principale di Feldman: il fatto che il commitment pubblico $c_0 = g^S$ possa rivelare informazioni sul segreto (specialmente se il segreto è piccolo).

Mentre Feldman è solo **computazionalmente** sicuro, Pedersen introduce un sistema **perfectly hiding**.

Il commitment di Feldman funziona in quanto omomorfico, infatti:

$$Commit(a+b) = Commit(a) \cdot Commit(b) \implies g^{a+b} = g^a \cdot g^b \mod p$$

Nello schema di Pederson, invece di usare un solo generatore g, ne vengono usati due, g e h, tali che nessuno conosca il logaritmo discreto di h rispetto a g ($\log_g(h)$).

Definizione (Pederson's Commitment). Per impegnarsi a un valore a, il mittente sceglie un numero casuale r e calcola:

$$Commit(a; r) = g^a h^r \mod p$$

Questo commitment è omomorfico, infatti:

Dimostrazione.

$$Commit(a+b;\ r_a + r_b) = g^{a+b} h^{r_a+r_b} = g^a h^{r_a} \cdot g^b h^{r_b} = Commit(a; r_a) \cdot Commit(b; r_b)$$

□

Proprietà: → $g^a h^r$ può essere commitment per ogni possibile valore

Perfecty Hiding Anche se hai una potenza di calcolo infinita, non puoi scoprire a perché, per ogni possibile valore a', esiste un valore r' che rende il commitment valido. Formalmente, $\forall a' \neq a \ \nexists! \ r'$ tale che → $\forall a' \neq a \ \exists! r'$

$$Commit(a'; r') = g^{a'} h^{r'} = g^a h^r = Commit(a; r)$$

Computationally Binding: Il mittente non può cambiare a dopo aver inviato il commitment, a meno di non saper risolvere il problema del logaritmo discreto (ovvero conoscere w tale che $h = g^w$)

Dimostrazione. Supponiamo per assurdo che si conosca $w = \log_g(h) \implies h = g^w$. Un utente ha creato un commitment per un valore a usando un valore casuale r,

$$Commit(a; r) = g^a h^r$$

Vuole ora trovare un nuovo valore r' che gli permetta di sostenere di essersi impegnato su un valore $a' \neq a$, ossia che valga l'uguaglianza

$$\begin{aligned} g^a h^r = g^{a'} h^{r'} &\implies g^a g^{wr} = g^{a'} g^{wr'} \\ &\implies g^{a+wr} = g^{a'+wr'} \\ &\implies a + wr = a' + wr' \mod q \\ &\implies r' = w^{-1}(a - a' + wr) \mod q \end{aligned}$$

→ q è t.c $2q+1 = p$ ⇒ $q = \frac{p-1}{2}$

Quindi, conoscendo w è possibile calcolare r' per **qualsiasi** a'. Questo significa che il commitment non vincola a nulla (nessun binding). □

Nota

Non è possibile garantire in contemporanea sia la *perfectly hiding* che la *perfectly binding*.

Come lo schema di Feldman, anche lo schema di Pedersen si articola di due figure chiave: il **Dealer** e il **Verifier**.

Dealer: Il Dealer genera due polinomi randomici $f(x)$ e $g(x)$ di grado $t-1$, dove il termine noto $f(0) = S$ e il termine noto $g(0) = r$.

$$\begin{aligned} f(x) &= S + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \\ g(x) &= r + b_1 x + b_2 x^2 + \cdots + b_{t-1} x^{t-1} \end{aligned}$$

Invia poi a ogni partecipante i una *doppia quota* $(x_i, y_i = f(x_i))$ e $(x_i, z_i = g(x_i))$. Infine, pubblica tutti i commitment calcolati come prima, ossia

$$\begin{aligned} c_0 &= g^S h^r \mod p \\ c_1 &= g^{a_1} h^{b_1} \mod p \\ &\vdots \\ c_{t-1} &= g^{a_{t-1}} h^{b_{t-1}} \mod p \end{aligned}$$

Verifier: Ogni partecipante i, ricevuta la sua *doppia quota* (x_i, y_i, z_i), può verificare se il Dealer è stato onesto confrontando la quota con i commit pubblici. Il partecipante contralla se la seguente uguaglianza è verifica:

$$\begin{aligned} c_0 \cdot c_1^{x_i} \cdot c_2^{x_i^2} \cdots c_{t-1}^{x_i^{t-1}} &= (g^S h^r) \cdot (g^{a_1} h^{b_1})^{x_i} \cdot (g^{a_2} h^{b_2})^{x_i^2} \cdots (g^{a_{t-1}} h^{b_{t-1}})^{x_i^{t-1}} \\ &= g^S \cdot g^{a_1 x_i} \cdot g^{a_2 x_i^2} \cdots g^{a_{t-1} x_i^{t-1}} \cdot h^r \cdot h^{b_1 x_i} \cdot h^{b_2 x_i^2} \cdots h^{b_{t-1} x_i^{t-1}} \\ &= g^{S + a_1 x_i + a_2 x_i^2 + \cdots + a_{t-1} x_i^{t-1}} \cdot h^{r + b_1 x_i + b_2 x_i^2 + \cdots + b_{t-1} x_i^{t-1}} \\ &= g^{f(x_i)} h^{g(x_i)} = g^{y_i} h^{z_i} \end{aligned}$$

Caratteristica	Feldman VSS	Pedersen VSS
Hiding (Privacy)	**Computazionale**: il valore g^s può rivelare informazioni se il segreto s è piccolo.	**Perfetto**: l'impegno $g^a h^r$ può corrispondere matematicamente a qualunque valore.
Binding (Vincolo)	**Perfetto**: matematicamente esiste un solo esponente possibile per un dato impegno pubblico.	**Computazionale**: il vincolo dipende dalla difficoltà di risolvere il logaritmo discreto $log_g h$.
Natura del segreto	Basato sulla generazione di un singolo polinomio casuale $p(x)$ di grado $t-1$.	Basato sulla generazione di due polinomi casuali $f(x)$ e $f'(x)$.

Tabella 4: Confronto tra gli schemi di Verifiable Secret Sharing di Feldman e Pedersen.

=) FELDMAN VSS
- HIDING -> COMPUTAZIONALE
- BINDING -> PERFETTO

PEDERSEN VSS
- HIDING -> PERFETTO
- BINDING -> COMPUTAZIONALE

ANCORA UN PROBLEMA
DOMANDA: CHI GENERA PUBBLICAMENTE h?
- IL DEALER? POTREBBE BARARE
- UN TTP?
- UNA PARTE STESSA?

DOMANDA 2: POSSIAMO GENERARE $h = g^w$ IN MODO CHE NESSUNO CONOSCA w?

→ COMPLETAMENTE DISTRIBUITO, NO TTP O AUTHORITY

5.3 Distributed Key Generation

Per garantire che lo schema di Pedersen sia effettivamente sicuro, dobbiamo risolvere l'ultimo grande ostacolo: la generazione del valore h. Se il Dealer generasse $h = g^w$ conoscendo w, potrebbe barare calcolando commitments che collidono e violando la proprietà di binding. Dobbiamo quindi trovare un modo per generare h senza che nessuno ne conosca la chiave privata w.

La soluzione a questo problema è la **Distributed Key Generation - DKG**, una tecnica che permette a un insieme di nodi di collaborare per creare una coppia di chiavi $(Pub_K, Priv_k)$, dove tutti conoscono la chiave pubblica $(h = g^w)$ e nessuno conosce quella privata (w).

Oltre alla VSS, la DKG è essenziale in molti sistemi crittografici moderni:

1. Sistemi basati su Logaritmo Discreto: Come El Gamal o la Crittografia Basata sull'Identità (IBE).
2. Rivelazione posticipata: Quando una chiave privata deve essere rivelata solo al verificarsi di condizioni speciali.
3. Funzioni distribuite: Quando la chiave privata non viene mai rivelata, ma è necessario calcolare funzioni che dipendono da essa. → ESTREMAMENTE IMP PER IBE

Il metodo più comune è il **Pedersen DKG (1991)**, che è essenzialmente un'applicazione ricorsiva dei concetti di Secret Sharing visti finora.

In questo schema, non esiste un'autorità centrale o un Trusted Third Party (TTP): ogni partecipante agisce contemporaneamente come "Dealer" di una propria parte del segreto e come "Parte" che riceve quote dagli altri.

Vediamo nel dettaglio come funziona per uno schema (t, n).

FASE 1: Generazione Locale e Polinomi Ogni partecipante i,genera un polinomio casuale $p_i(x)$ di grado $t-1$, dove il termine noto è il segreto $S_i = a_{0,i}$

$$p_i(x) = S_i + a_{1,i}x + a_{2,i}x^2 + \cdots + a_{t-1,i}x^{t-1}$$

FASE 2: Scambio delle Quote Ogni partecipante distribuisce le quote del proprio segreto agli altri. Quindi il partecipante i riceve $n-1$ quote e distribuisce a sua volta $n-1$ quote del proprio segreto.

FASE3: Calcolo Quota Finale e Commitiments Ora ogni partecipante somma ciò che ha ricevuto per ottenere la sua quota del segreto globale:

$$y_i = \sum_{j=1}^{n} p_j(i)$$

Infine, ogni parte pubblica l'impegno del proprio segreto locale $c_i = g^{a_{0,i}}$.

In conclusione, il segreto globale $S = \sum_{i=1}^{n} a_{0,i}$ non è noto a nessuno, poiché ogni $a_{0,i}$ resta privato. La chiave pubblica associata è g^S, calcolabile da tutti come

$$g^S = \prod_{i=1}^{n} g^{a_{0,i}} = \prod_{i=1}^{n} c_i.$$

Infine, ogni partecipante può verificare la correttezza della propria quota rispetto ai commitment pubblici divulgati durante la DKG.

Quindi per poter utilizzare lo schema di **Pedersen VSS** in modo sicuro, è necessario che il parametro pubblico h sia generato tramite il protocollo **Pedersen DKG**.

Fase 0 (DKG): I partecipanti collaborano per generare $h = g^w$ senza che w venga rivelato a nessuno.

Fase 1 (Commitment): Si usa questo h per creare impegni ultra-sicuri del tipo $g^a h^r$.

Fase 2 (VSS): Il Dealer distribuisce le quote del segreto usando i due polinomi $f(x)$ e $g(x)$, permettendo a tutti di verificare l'onestà tramite gli impegni basati su g e h.

Esempio. In questo scenario, 4 partecipanti ($n = 4$) collaborano per generare una chiave pubblica comune con una soglia di ricostruzione di 3 persone ($t = 3$).

1. Generazione dei Segreti Locali e dei Polinomi

Ogni partecipante P_i sceglie un proprio segreto casuale $a_{0,i}$ e definisce un polinomio di grado 2 $(t-1)$:

- P_1 sceglie $a_{0,1}$ e crea $p_1(x) = a_{0,1} + a_{1,1}x + a_{2,1}x^2$
- P_2 sceglie $a_{0,2}$ e crea $p_2(x) = a_{0,2} + a_{1,2}x + a_{2,2}x^2$
- P_3 sceglie $a_{0,3}$ e crea $p_3(x) = a_{0,3} + a_{1,3}x + a_{2,3}x^2$
- P_4 sceglie $a_{0,4}$ e crea $p_4(x) = a_{0,4} + a_{1,4}x + a_{2,4}x^2$

2. Scambio Sicuro delle Quote

Ogni partecipante calcola i valori del proprio polinomio nei punti $1, 2, 3, 4$ e li invia agli altri. Ad esempio, P_1 tiene per sé $p_1(1)$ e invia:

- $p_1(2) \to P_2, \quad p_1(3) \to P_3, \quad p_1(4) \to P_4$

Contemporaneamente, P_1 riceve le quote dagli altri partecipanti: $p_2(1)$, $p_3(1)$ e $p_4(1)$.

3. Calcolo della Quota Finale (Full Share)

Ogni partecipante somma tutte le quote ricevute (inclusa la propria) per ottenere la sua quota del segreto collettivo s:

- P_1 calcola $y_1 = p_1(1) + p_2(1) + p_3(1) + p_4(1)$
- P_2 calcola $y_2 = p_1(2) + p_2(2) + p_3(2) + p_4(2)$
- P_3 calcola $y_3 = p_1(3) + p_2(3) + p_3(3) + p_4(3)$
- P_4 calcola $y_4 = p_1(4) + p_2(4) + p_3(4) + p_4(4)$

4. Pubblicazione degli Impegni (Commitments)

Per permettere la verifica e il calcolo della chiave pubblica, ogni partecipante pubblica l'impegno del proprio segreto:

- Broadcast di: $g^{a_{0,1}}, g^{a_{0,2}}, g^{a_{0,3}}, g^{a_{0,4}}$

5. Conclusione: Il Segreto Condiviso

Senza che nessuno conosca il segreto totale s, il gruppo ha generato:

- **Segreto Globale (Privato):** $s = a_{0,1} + a_{0,2} + a_{0,3} + a_{0,4}$
- **Chiave Pubblica:** $g^s = g^{a_{0,1}} \cdot g^{a_{0,2}} \cdot g^{a_{0,3}} \cdot g^{a_{0,4}}$
- **Quote di Verifica:** Ogni y_i è una quota valida del segreto s

6 Threshold and Policy-Based Cryptography

In questo capitolo estendiamo il concetto di **secret sharing**: l'obiettivo non è solo custodire un segreto, ma distribuire l'autorità di eseguire operazioni crittografiche (decifratura e firma) tra più entità, aumentando la resilienza e riducendo i punti di fallimento.

Threshold Encryption: Adattiamo **ElGamal** in forma threshold, in modo che la decifratura sia possibile solo con la cooperazione di almeno t partecipanti.

Threshold Signatures: Permettiamo a un gruppo di generare collettivamente una firma standard e verificabile, ricostruibile solo da insiemi di almeno t partecipanti.

Policy-Based Schemes: Generalizziamo le soglie a strutture di accesso (politiche) più ricche, dove insiemi autorizzati di partecipanti possono decifrare/firmare secondo regole prefissate.

6.1 Threshold Encryption: ElGamal

L'algoritmo di cifratura di ElGamal trasforma lo scambio di chiavi di Diffie-Hellman in un vero sistema di cifratura. Vediamo il caso del logaritmo discreto (più avanti lo vederemo con le curve ellittiche).

ElGamal Classico

Il sistema di cifratura classico di ElGamal funziona in questo modo: innanzitutto si scelgono un numero p primo molto grande e un generatore g del gruppo ciclico.

Generazione delle chiavi: Si genera la coppia (SK, PK).

$$\begin{aligned} SK &\leftarrow_{u.a.r.} \mathbb{Z}_p^* \\ PK &= g^{SK} \pmod p \end{aligned}$$

Cifratura: Dato $m \in \mathbb{Z}_p^*$, si sceglie $k \leftarrow_{u.a.r.} \mathbb{Z}_p^*$ (*chiave effimera*) e si calcolano:

$$\begin{aligned} A &= g^k \pmod p \\ B &= m \cdot PK^k \pmod p \end{aligned}$$

Il cifrato è la coppia (A, B).

Decifratura: Ricevuto (A, B), si ricava m usando SK:

$$\begin{aligned} m &= B \cdot (A^{SK})^{-1} \pmod p \\ &= \frac{B}{A^{SK}} = \frac{m \cdot g^{k\,SK}}{g^{k\,SK}} \pmod p \end{aligned}$$

Threshold ElGamal

Il sistema classico di cifratura di ElGamal si può estendere in un sistema **Threshold (a soglia)**. L'idea è che vogliamo cifrare un messaggio in modo che nessuno possa leggerlo da solo, ma sia necessaria la collaborazione di almeno t partecipanti su un totale di n.

Setup: Si condivide la chiave privata SK tramite **Shamir Secret Sharing**. Si fissa un campo $\mathbb{F}_q$ con $q > n$ e si scelgono indici pubblici distinti $x_1, \ldots, x_n \in \mathbb{F}_q^*$. Si genera un polinomio $f(x)$ di grado $t-1$ con termine noto $f(0) = SK$, e si consegna a ciascun partecipante i la quota $SK_i = f(x_i)$. Per evitare una TTP e impedire che qualcuno conosca SK per intero, si usa **Pedersen DKG/VSS**.

Si lavora in $\mathbb{Z}_p^*$ con generatore g di un sottogruppo di ordine q e si pubblica $PK = g^{SK} \pmod p$.

Cifratura: Il mittente cifra normalmente con PK. Sceglie $k \leftarrow_{u.a.r.} \mathbb{Z}_q$ e calcola

$$(A, B) = (g^k \pmod p,\ m \cdot PK^k \pmod p).$$

S QUANDO LECESSARIO; MA QUESTO FUNZIONAVA UNA SOLA VOLTA => UNA VOLTA RICOSTRUITA S TUTTO POTEVA ESSERE RICOSTRUITO

Decifratura collaborativa: Almeno t partecipanti cooperano senza ricostruire SK. Ogni i calcola la propria **decifratura parziale** $S_i = A^{SK_i} \pmod p$ e la invia al combinatore.

Ricostruzione del messaggio: Il combinatore esegue l'interpolazione di Lagrange **all'esponente** (con pesi in $\mathbb{F}_q$):

$$A^{SK} = \prod_{i=1}^{t} S_i^{\lambda_i} \pmod p, \qquad \lambda_i = \prod_{\substack{j=1 \\ j\neq i}}^{t} \frac{-x_j}{x_i - x_j} \in \mathbb{F}_q.$$

Ottenuto A^{SK}, decifra

$$m = B \cdot \left(A^{SK}\right)^{-1} \pmod p.$$

6.2 Threshold Signatures: RSA

Il **Threshold RSA** permette a un gruppo di n entità di produrre congiuntamente una firma, richiedendo la cooperazione di almeno t partecipanti. A differenza di ElGamal, l'adattamento a soglia di RSA è più complesso: l'interpolazione di Lagrange richiede inversioni in $\mathbb{Z}_{\phi(N)}$, ma $\phi(N)$ non deve essere conosciuta (rivelerebbe i fattori di N) e, inoltre, $\mathbb{Z}_{\phi(N)}$ non è in generale un campo.

6.2.1 RSA Classico

Si scelgono due primi grandi p, q, si calcola $N = pq$ e $\phi(N) = (p-1)(q-1)$. Si sceglie e coprimo con $\phi(N)$ e si computa $d = e^{-1} \bmod \phi(N)$.

Firma: Dato un messaggio m, si calcola $\sigma = h(m)^d \bmod N$.

Verifica: Si accetta se $\sigma^e \equiv h(m) \pmod N$.

6.2.2 Threshold RSA

Il nodo critico in Threshold RSA è la combinazione corretta delle firme parziali. Idealmente, il partecipante i dovrebbe contribuire con

$$h(m)^{\,d_i\,\lambda_i} \pmod N,$$

dove i pesi di Lagrange sono

$$\lambda_i = \prod_{\substack{j=1 \\ j\neq i}}^{t} \frac{-x_j}{x_i - x_j} \qquad \text{e} \qquad \lambda_i = \frac{\alpha_i}{\beta_i} \text{ con } \gcd(\alpha_i, \beta_i) = 1.$$

Per ottenere l'esponente frazionario si richiederebbe $\beta_i^{-1} \bmod \phi(N)$:

$$h(m)^{\,d_i\,\lambda_i} = h(m)^{\,d_i\,\alpha_i\,\beta_i^{-1}} \pmod N,$$

DIVERSE SE β È PARI, L'INVERSO NEMMENO ESISTE

ma calcolare $\beta_i^{-1} \bmod \phi(N)$ non è possibile senza conoscere $\phi(N)$ (che rivelerebbe i fattori di N).

Osservazione: se si scelgono indici $x_1, \ldots, x_n \in \{1, \ldots, n\}$, i denominatori dei λ_i dividono $n!$. Si può quindi definire un denominatore comune $\Delta = n!$ per *ripulire* le frazioni. → AKA EVITARE LE INVERSE

Generazione distribuita: Si genera congiuntamente $N = pq$ senza rivelare p, q e si condivide l'esponente segreto d tramite Shamir/VSS: ogni partecipante i riceve una quota $y_i = f(x_i)$ di d. Si fissano indici pubblici $x_1, \ldots, x_n \in \{1, \ldots, n\}$ e si pone un denominatore comune $\Delta = n!$. I pesi di Lagrange in 0 sono

$$\lambda_i = \prod_{\substack{j=1 \\ j\neq i}}^{t} \frac{-x_j}{x_i - x_j} = \frac{\alpha_i}{\beta_i}, \qquad \beta_i \mid \Delta,$$

e definiamo $\Lambda_i = \Delta\,\lambda_i \in \mathbb{Z}$.

Firme parziali: Ogni partecipante i pubblica

$$\tau_i = h(m)^{y_i \Lambda_i} \mod N,$$

equivalente a calcolare $h(m)^{y_i}$ e poi elevarlo alla potenza intera Λ_i.

Aggregazione: Il combinatore calcola il prodotto

$$T = \prod_{i=1}^{t} \tau_i \equiv h(m)^{\Delta \sum_{i=1}^{t} y_i \lambda_i} \equiv h(m)^{\Delta d} \pmod N.$$

↳ ABBIAMO EVITATO LE INVERSE MA DOBBIAMO SOLVE UNA CHIAVE $\Delta = m!$

↳ USIAMO L'ATTACCO RSA COMMON MODULUS (*)

Rimozione del fattore Δ (Bézout): Poiché tipicamente $\gcd(e, \Delta) = 1$, si trovano $a, b \in \mathbb{Z}$ tali che $ae + b\Delta = 1$ (Algoritmo Euclideo Esteso). La firma è

RSA COMMON MODULUS →

$$\begin{aligned}\sigma = (h(m)^e)^a \cdot T^b \mod N &= (h(m)^e)^a \cdot h(m)^{b\Delta d} \mod N \\ &= h(m)^{ea+b\Delta} \cdot h(m)^d \mod N \\ &= h(m)^1 \cdot h(m)^d \mod N \\ &= h(m)^d \mod N\end{aligned}$$

↳ $\sigma = h(m)^a T^b$

$= (h(m)^{ed})^a T^b$

$= (h(m)^{ed})^a h(m)^{ab}$

$= h(m)^{d(ea+\Delta b)}$

$= h(m)^d \mod N$

Verifica: Come in RSA classico: $\sigma^e \equiv h(m) \pmod N$.

EL GAMAL

QUI LO ABBIAMO VISTO IMPLEMENTATO SUI MODULI, MA NELLA REALTÀ È PIÙ CONVENIENTE IMPLEMENTARLO SULLE CURVE ELLITTICHE

PERCHÉ CONVIENE DI PIÙ? PERCHÉ LE E.C SCALANO MEGLIO DEI MODULI

CHIAVE SIMM	GRANDEZZA MODULO	E.C
80 bits	1024 bits	163 bits
128 bits	3072 bits	283 bits
256 bit	15360 bits	521 bits

(*) RSA COMMON MODULUS ATTACK → VICTOR SHOUP

SCENARIO

- ALICE CIFRA IL MSG M CON LA CHIAVE e_A
- BOB CIFRA LO STESSO MSG M CON e_B
- IL MODULO N È LO STESSO $\Rightarrow \gcd(e_A, e_B) = 1 \Rightarrow e_A$ E e_B COPRIMI

ATTACCO

IN QUESTO SCENARIO, PER RICAVARE M È SUFF. APPLICARE L'ALGORITMO EUCLIDEO ESTESO PER TROVARE r, s T.C $e_A \cdot r + e_B \cdot s = 1 = \gcd(e_A, e_B)$

UNA VOLTA CALCOLATI r E s, IL MSG M VIENE DECIFRATO (SENZA USARE LA CHIAVE PRIVATA) COME

$$(M^{e_A})^r \cdot (M^{e_B})^s = M^{e_A \cdot r + e_B \cdot s} = M^1 = M$$

LO ABBIAMO USATO NEL THRES. RSA PERCHÉ AVEVAMO: $\begin{cases} \cdot\, m! = \Delta \\ \cdot\, H(m)^{d \cdot \Delta} \end{cases}$ MA VOLEVAMO $H(m)^d = y$; ESSENDO CHE ABBIAMO ANCHE $H(m)^{d\Delta} = y^\Delta$ E $H(m) = H(m)^{de} = y^e$ ALLORA, SE e E Δ SONO COPRIMI POSSIAMO APPLICARE L'ATTACCO E OTTENERE $H(m)^d = y$.

SE e È UN PRIMO $> n \Rightarrow$ QUESTO SIGNIFICHE FUNZIONA

LEVANDO QUI, LO SCHEMA DI FIRME RSA MINIME PERFETTAMENTE SICURO

7 Linear Secret Sharing & Access Control Matrix

In questa sezione reinterpretiamo lo schema di Shamir in forma lineare e arriviamo alla sua generalizzazione: il Linear Secret Sharing (LSSS). L'obiettivo è condividere un segreto S tra più partecipanti in modo che solo i sottoinsiemi autorizzati (la struttura di accesso) possano ricostruirlo.

Shamir (t, n) su $\mathbb{F}_q$ assegna a ogni partecipante i la quota

$$y_i = f(x_i) = S + a_1 x_i + a_2 x_i^2 + \cdots + a_{t-1} x_i^{t-1},$$

che può essere vista come prodotto scalare

$$y_i = \underbrace{(1,\ x_i,\ x_i^2,\ \ldots,\ x_i^{t-1})}_{\text{riga } i} \cdot \underbrace{(S,\ a_1,\ a_2,\ \ldots,\ a_{t-1})^T}_{\text{vettore dei coefficienti}}.$$

- MAT DI VANDERMONDE MAT LE CUI RIGHE (O COLONNE) HANNO ELEMENTI, A PARTIRE DA 1, IN PROGRESSIONE GEOMETRICA $a_{i,j} = a_i^{j-1}$

Ponendo $r_j = a_j$ si ha

$$\begin{pmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{t-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{t-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^{t-1} \end{pmatrix} \cdot \begin{pmatrix} S \\ r_1 \\ \vdots \\ r_{t-1} \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix},$$

cioè un sistema lineare $A\mathbf{x} = \mathbf{y}$ con A di tipo Vandermonde, $\mathbf{x} = (S, r_1, \ldots, r_{t-1})$ e $\mathbf{y} = (y_1, \ldots, y_n)$.

Se un gruppo di t partecipanti si riunisce, può formare un sistema lineare quadrato utilizzando solo le proprie righe e le proprie quote. Se chiamiamo A' la sottomatrice $t \times t$ dei partecipanti presenti, il segreto si trova risolvendo:

$$\mathbf{x} = (A')^{-1} \mathbf{y}_{presenti}$$

Il segreto S è semplicemente la prima entrata del vettore risultato.

- ES SCHEMA (3, 4)

$A = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix}$, $\bar{x} = \begin{pmatrix} S \\ r_1 \\ r_2 \end{pmatrix}$, $\bar{y} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$

⇒ DOBBIAMO RISOLVERE $\bar{x} = A^{-1}\bar{y}$ E PRENDERE x_0 (PRIMO ELEMENTO DI $\bar{x}$)

IL SISTEMA DIVENTA

$\begin{pmatrix} S \\ r_1 \\ r_2 \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix}^{-1} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$

7.1 LSSS e MSP

Per semplicità, consideriamo uno schema $(3, 4)$:

$$\begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \end{pmatrix} \cdot \begin{pmatrix} S \\ r_1 \\ r_2 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix}.$$

Per ricostruire S usando le sole quote di P_1, P_2, P_4 si risolve

$$\begin{pmatrix} S \\ r_1 \\ r_2 \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_4 & x_4^2 \end{pmatrix}^{-1} \cdot \begin{pmatrix} y_1 \\ y_2 \\ y_4 \end{pmatrix}.$$

Interpretiamo ora le righe dei partecipanti come vettori:

$$\mathbf{v_1} = (1, x_1, x_1^2), \quad \mathbf{v_2} = (1, x_2, x_2^2), \quad \mathbf{v_4} = (1, x_4, x_4^2).$$

Questi tre vettori generano uno spazio di dimensione 3. Poiché il segreto S è associato alla prima colonna, per eliminare la casualità r_1, r_2 cerchiamo coefficienti c_1, c_2, c_3 tali che

$$c_1\mathbf{v_1} + c_2\mathbf{v_2} + c_3\mathbf{v_4} = (1, 0, 0).$$

Allora

$$S = (1, 0, 0) \cdot (S, r_1, r_2)^T = c_1 y_1 + c_2 y_2 + c_3 y_4$$

$$\Rightarrow \frac{x_2 x_4}{(x_1 - x_2)(x_1 - x_4)} \cdot y_1 + \frac{x_1 x_4}{(x_2 - x_1)(x_2 - x_4)} \cdot y_2 + \frac{x_1 x_2}{(x_4 - x_1)(x_4 - x_2)} \cdot y_3$$

dove i coefficienti c_i si ottengono risolvendo

$$\begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_4 & x_4^2 \end{pmatrix} \cdot \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

$$\Rightarrow \begin{cases} c_1 = \frac{x_2 x_4}{(x_1-x_2)(x_1-x_4)} \\ c_2 = \frac{x_1 x_4}{(x_2-x_1)(x_2-x_4)} \\ c_4 = \frac{x_1 x_2}{(x_4-x_1)(x_4-x_2)} \end{cases}$$

In sintesi, mentre Shamir usa matrici di Vandermonde, la vista lineare consente di usare una qualunque matrice per codificare chi può ricostruire il segreto. → SHAMIR SECRET SHARING MATRICE DI CUI PUOI "SPAN"

Definizione (LSSS). Un **Linear Secret Sharing Scheme** con soglia t su un campo $\mathbb{F}_q$ condivide un segreto S usando sole trasformazioni lineari.

Si fissa una matrice $A \in \mathbb{F}_q^{n \times t}$ arbitraria e si costruisce il vettore

$$\mathbf{x} = (S, r_1, \dots, r_{t-1})^T \in \mathbb{F}_q^t.$$

Le quote distribuite sono il prodotto

$$\mathbf{y} = A\mathbf{x} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \qquad \text{Quota assegnata a } P_i: \quad y_i = \mathbf{a}_i \cdot \mathbf{x} = a_{i,1} S + \sum_{j=2}^{t} a_{i,j}\, r_{j-1},$$

TEOREMA ANOS BEIMEL DICE CHE LSSS = MSP

dove $\mathbf{a}_i = (a_{i,1}, \dots, a_{i,t})$ è la i-esima riga di A.

Per un insieme autorizzato U esistono coefficienti $c_i \in \mathbb{F}_q$ tali che → INSIEME U PER MSP

$$\sum_{i \in U} c_i\, \mathbf{a}_i = (1, 0, \dots, 0) \quad \Rightarrow \quad S = \sum_{i \in U} c_i\, y_i.$$

Questo concetto è conosciuto anche come **MSP - Monotone Span Program**. Ossia l'insieme U è considerato autorizzato se e solo se le righe della matrice A in loro possesso riescono a generare (fare lo **span**) il **vettore target** $(1, 0, 0, \dots)$ attraverso una combinazione lineare.

Il programma è definito "monotono" perché se un insieme di partecipanti può ricostruire il segreto, anche qualsiasi insieme più grande che li includa può farlo. Non è possibile invece includere negazioni (es. *A e B, ma non C*).

Esempio. Il **Trivial Secret Sharing** è un LSSS. Consideriamo uno schema $(3, 3)$: le quote si ottengono come

$$\begin{pmatrix} 1 & -1 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} S \\ r_1 \\ r_2 \end{pmatrix} = \begin{pmatrix} S - r_1 - r_2 \\ r_1 \\ r_2 \end{pmatrix},$$

cioè $y_1 = S - r_1 - r_2$, $y_2 = r_1$, $y_3 = r_2$.

Per la ricostruzione, lo span program cerca c_1, c_2, c_3 tali che

$$c_1(1, -1, -1) + c_2(0, 1, 0) + c_3(0, 0, 1) = (1, 0, 0).$$

Una scelta valida è $c_1 = c_2 = c_3 = 1$, da cui

$$S = c_1 y_1 + c_2 y_2 + c_3 y_3 = (S - r_1 - r_2) + r_1 + r_2 = S.$$

LSSS è omomorfico Siano S_a e S_b due segreti condivisi con la stessa matrice di distribuzione A. Definiamo

$$\mathbf{x_a} = (S_a, r_{1a}, \dots, r_{t-1,a})^T, \qquad \mathbf{x_b} = (S_b, r_{1b}, \dots, r_{t-1,b})^T,$$

e le rispettive quote $\mathbf{y_a} = A\mathbf{x_a}$, $\mathbf{y_b} = A\mathbf{x_b}$. Per linearità,

$$\mathbf{y_a} + \mathbf{y_b} = A\mathbf{x_a} + A\mathbf{x_b} = A(\mathbf{x_a} + \mathbf{x_b}) = A\,(S_a + S_b,\ r_{1a} + r_{1b}, \dots, r_{t-1,a} + r_{t-1,b})^T$$

Ne segue che, per ogni partecipante i, $y_{i,a} + y_{i,b}$ è una quota valida del segreto $S_a + S_b$ rispetto ad A.

Esempio (MSP). Riscriviamo un MSP su $GF(2)$ (Galois Field, dove l'addizione = XOR e la moltiplicazione = AND). Le righe della matrice A sono etichettate dai partecipanti:

$$\begin{array}{c} P_2 \\ P_2 \\ P_1 \\ P_3 \\ P_4 \end{array} \begin{pmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} = A, \qquad \mathbf{t} = (0,0,0,1).$$

Un sottoinsieme è autorizzato se le righe corrispondenti generano il vettore target $\mathbf{t}$. Poiché P_2 compare due volte, riceve due quote (due righe).

Dimostriamo che $\{P_2, P_4\}$ è autorizzato. Indicando

$$\mathbf{r}_{2,1} = (1,0,1,1), \quad \mathbf{r}_{2,2} = (0,1,1,0), \quad \mathbf{r}_4 = (1,1,0,0),$$

in $GF(2)$ vale

$$\mathbf{r}_{2,1} \oplus \mathbf{r}_{2,2} \oplus \mathbf{r}_4 = (1,0,1,1) \oplus (0,1,1,0) \oplus (1,1,0,0) = (0,0,0,1) = \mathbf{t}.$$

Quindi P_2 (con entrambe le sue quote) insieme a P_4 possono ricostruire il segreto.

Viceversa, si può dimostrare che non esiste alcuna combinazione lineare ottenibile usando solo le righe di P_1 e P_2; pertanto l'insieme $\{P_1, P_2\}$ non è autorizzato a ricostruire il segreto.

Esempio (LSSS). Consideriamo lo stesso esempio precedente e mostriamo l'equivalenza MSP-LSSS. Usiamo la stessa matrice A (righe etichettate dai partecipanti) e lavoriamo su $GF(2)$. Scegliamo

$$\mathbf{x} = (r_1, r_2, r_3, S)^T \quad \text{così che} \quad \mathbf{t} = (0,0,0,1) \;\Rightarrow\; \mathbf{t} \cdot \mathbf{x} = S.$$

$$\begin{array}{c} P_2 \\ P_2 \\ P_1 \\ P_3 \\ P_4 \end{array} \begin{pmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ S \end{pmatrix} = \begin{pmatrix} r_1 + r_3 + S \\ r_2 + r_3 \\ r_2 + r_3 \\ r_3 + S \\ r_1 + r_2 \end{pmatrix} = \begin{pmatrix} y_{2,1} \\ y_{2,2} \\ y_1 \\ y_3 \\ y_4 \end{pmatrix}.$$

Poiché, in $GF(2)$, $\mathbf{r}_{2,1} \oplus \mathbf{r}_{2,2} \oplus \mathbf{r}_4 = \mathbf{t}$, la combinazione corrispondente delle quote è

$$y_{2,1} \oplus y_{2,2} \oplus y_4 = (r_1 + r_3 + S) \oplus (r_2 + r_3) \oplus (r_1 + r_2) = S.$$

Ricordare che in $GF(2)$ la somma coincide con lo XOR, quindi "$+$" = "$\oplus$".

7.2 LSSS e Strutture di Accesso

Gli LSSS permettono di implementare politiche di accesso monotone: un sottoinsieme autorizzato resta autorizzato aggiungendo altri partecipanti. Formalmente, data l'insieme dei partecipanti $\mathcal{P} = \{P_1, \ldots, P_n\}$, una struttura di accesso è $A \subseteq 2^{\mathcal{P}}$ tale che

A SIMILE A INS. DELLE PARTI DI $\mathcal{P}$ SENZA SINGLETON

$$U \in A \text{ e } U \subseteq V \;\Rightarrow\; V \in A.$$

È utile descrivere A tramite gli insiemi minimali autorizzati $M(A)$ (quelli in A per cui nessun sottoinsieme proprio è autorizzato); A è poi la chiusura monotona di $M(A)$.

Esempio. Sia $\mathcal{P} = \{P_1, P_2, P_3, P_4\}$ e $M(A) = \{\{P_1, P_2\}\}$. Allora

$$A = \{\{P_1, P_2\}, \{P_1, P_2, P_3\}, \{P_1, P_2, P_4\}, \{P_1, P_2, P_3, P_4\}\}.$$

CONSEGUENZA: OGNI PREDICATO BOOLEANO (SENZA NEGAZIONE) PUÒ ESSERE SUPPORTATO

Esempio. La politica $P_1 \wedge (P_2 \vee (P_3 \wedge P_4))$, ossia P_1 deve essere sempre presente, insieme a P_2 oppure alla coppia P_3 e P_4 ed ha insiemi minimali

$$M(A) = \{\{P_1, P_2\},\ \{P_1, P_3, P_4\}\},$$

e A contiene tutti i loro superinsiemi (es. $\{P_1, P_2, P_3\}$, $\{P_1, P_2, P_4\}$, $\{P_1, P_2, P_3, P_4\}$, ecc.).

Solo formule monotone (AND/OR) sono supportate; la negazione (NOT) non è ammessa.

Costruzione della matrice di accesso da un predicato logico

Rappresentiamo la formula monotona come un albero (porte AND/OR) e propaghiamo un vettore lungo l'albero fino alle foglie. Le foglie etichettate dai partecipanti diventano le righe della matrice A.

Regole di propagazione per un vettore $\mathbf{v} \in \mathbb{F}_q^d$:

OR La porta OR inoltra lo stesso vettore a tutti i figli: ogni figlio riceve $\mathbf{v}$ (nessuna variazione di dimensione).

AND La porta AND richiede cooperazione fra i rami. Si introduce una nuova coordinata e si incrementa la dimensione a $d+1$:

$$\text{figlio sinistro}: (\mathbf{v}, 1), \qquad \text{figlio destro}: (\mathbf{0}_d, -1).$$

La combinazione lineare delle due parti annulla la nuova coordinata e ricostruisce $\mathbf{v}$.

QUANDO SI HA AND A 3 SI METTE IL PADDING AL CENTRO
ES:
$(A \wedge C \wedge D) \vee (B \wedge C)$

A 1 -1 -1 0
C 0 1 0 0
D 0 0 1 0
B 1 0 0 -1
C 0 0 0 1

↳ PAD. AL CENTRO PERCHÉ RITORNO CHE FINE

↳ $(v,1) + (0,-1) = (v,0)$

AND A 3 PARTI

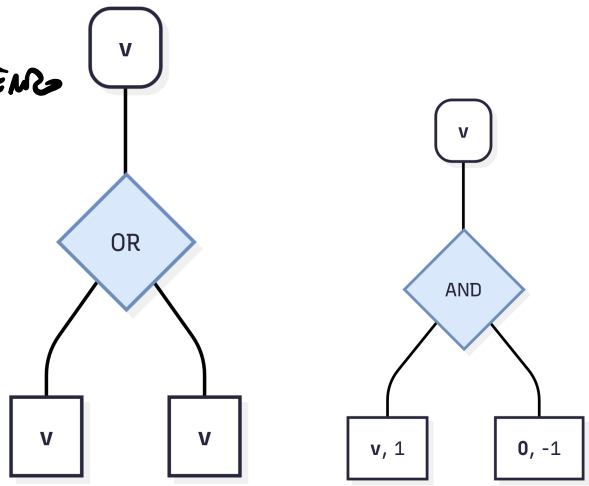


Figura 3: Porte OR e AND

Si parte dalla radice con il *vettore target* $\mathbf{t} = (1, 0, \ldots, 0)$ (prima coordinata associata al segreto S) e si applicano le regole fino alle foglie. I vettori alle foglie costituiscono le righe di A; un insieme di partecipanti è autorizzato se le sue righe generano $\mathbf{t}$.

↳SPOILER: L'INSIEME AUTORIZZATO È PARI AD $M(A)$, OVVERO IN QUESTO ESEMPIO L'INSIEME AUTORIZZATO SARÀ $\{\{P_1, P_2\}, \{P_1, P_2, P_3\}\}$

Esempio. Riprendiamo l'esempio $P_1 \wedge (P_2 \vee (P_3 \wedge P_4))$ e costruiamo la matrice di accesso.

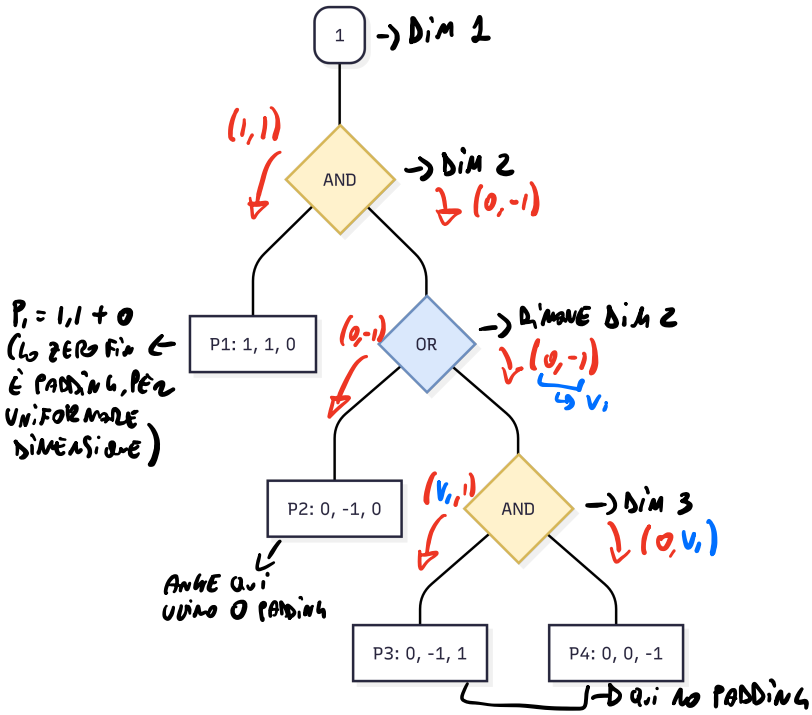


Figura 4: Albero costruito a partire dalla formula logica.

Propagando il vettore target $\mathbf{t} = (1)$ e uniformando la dimensione a 3 si ottengono i vettori alle foglie:

$$\begin{array}{c} P_1 \\ P_2 \\ P_3 \\ P_4 \end{array} \begin{pmatrix} 1 & 1 & 0 \\ 0 & -1 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & -1 \end{pmatrix} = A, \qquad \mathbf{t} = (1, 0, 0).$$

Condividendo $\mathbf{x} = (S, r_1, r_2)^T$, le quote sono

$$y_1 = S + r_1, \quad y_2 = -r_1, \quad y_3 = -r_1 + r_2, \quad y_4 = -r_2.$$

Ricostruzione (insiemi minimali autorizzati):

$$\{P_1, P_2\}: \; S = y_1 + y_2, \qquad \{P_1, P_3, P_4\}: \; S = y_1 + y_3 + y_4.$$

3) ALLA FINE AGGIUNGERE PUNTO $\mathcal{O}$ (ALL'INF.)
$\Rightarrow E(E_5) = \{\mathcal{O}, (0,1), (0,4), (2,1), (2,4), (3,1), (3,4), (4,2), (4,3)\}$

8 Elliptic Curve

La crittografia moderna fa ampio uso delle curve ellittiche per motivi di efficienza: a parità di sicurezza, le chiavi sono molto più piccole rispetto a RSA o agli schemi basati su $\mathbb{Z}_p^*$. Ad esempio, una chiave ECC da 256 bit è comparabile a una chiave RSA da 3072 bit; la crescita della sicurezza in ECC richiede chiavi di dimensione circa lineare, mentre in RSA è sostanzialmente esponenziale.

Definizione (Curva ellittica). Sia $p > 3$ un primo. Una curva ellittica $E_p(x, y)$ definita sul campo finito $\mathbb{F}_p$ è data (forma di Weierstrass ridotta) da

$$y^2 \equiv x^3 + ax + b \pmod{p},$$

con $a, b \in \mathbb{F}_p$ tali che $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$ (discriminante non nullo). Questa condizione garantisce che l'equazione $x^3 + ax + b = 0$ non ammetta doppia radice.

L'insieme dei punti $E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : \ y^2 \equiv x^3 + ax + b\} \cup \{\mathcal{O}\}$ forma un gruppo abeliano rispetto alla somma tra punti, dove $\mathcal{O}$ è il punto all'infinito (elemento neutro).

Esempio. Consideriamo la curva $E_5(x, y)$ definita su $\mathbb{Z}_5$, dall'equazione

↳ vedi qua (a)

$$y^2 = x^3 + x + 1$$

Stiamo quindi considerando tutti quei punti

$$\begin{aligned} E(\mathbb{Z}_5) &= \{(x, y) \in \mathbb{Z}_5^2 : \ y^2 \equiv x^3 + x + 1 \pmod{5}\} \cup \{\mathcal{O}\} \\ &= \{\mathcal{O}, (0,1), (0,4), (2,1), (2,4), (3,1), (3,4), (4,2), (4,3)\} \end{aligned}$$

Definizione (La legge di addizione). Sia $E/\mathbb{F}_p$ (con $p > 3$) definita da $y^2 \equiv x^3 + ax + b \pmod{p}$.

Il punto all'infinito $\mathcal{O}$ è l'elemento neutro:

$$\forall P \in E(\mathbb{F}_p), \quad P + \mathcal{O} = \mathcal{O} + P = P.$$

L'operazione di gruppo "+" sui punti di $E(\mathbb{F}_p)$ è definita come segue. Siano $P = (x_1, y_1)$ e $Q = (x_2, y_2)$ due punti, allora

Se $\mathbf{x_1 \neq x_2}$: Ponendo $\lambda \equiv (y_2 - y_1)(x_2 - x_1)^{-1} \pmod{p}$, si ha

$$x_3 \equiv \lambda^2 - x_1 - x_2 \pmod{p}, \qquad y_3 \equiv \lambda(x_1 - x_3) - y_1 \pmod{p}.$$

$\} P \neq Q \Rightarrow P + Q = R$

Allora $P + Q = R = (x_3, y_3)$.

Se $\mathbf{P = Q = (x_1, y_1)}$ con $\mathbf{y_1 \neq 0}$: Ponendo $\lambda \equiv (3x_1^2 + a)(2y_1)^{-1} \pmod{p}$, si ha

$$x_3 \equiv \lambda^2 - 2x_1 \pmod{p}, \qquad y_3 \equiv \lambda(x_1 - x_3) - y_1 \pmod{p}.$$

$\} P = Q \Rightarrow P + P = 2P$

Allora $2P = R = (x_3, y_3)$.

Se $\mathbf{x_1 = x_2}$ e $\mathbf{y_1 \equiv -y_2 \pmod{p}}$: $P + Q = \mathcal{O}$.

Se $\mathbf{P = Q}$ e $\mathbf{y_1 = 0}$: $2P = \mathcal{O}$.

Questa legge di addizione rende l'insieme $E(\mathbb{F}_p)$ un gruppo. L'elemento identità è il punto all'infinito $\mathcal{O}$. Ogni punto $P = (x_1, y_1) \in E(\mathbb{F}_p)$, con $P \neq \mathcal{O}$, ha inverso additivo $-P = (x_1, -y_1)$. Inoltre si può dimostrare che la legge è associativa. La legge è chiaramente commutativa, $P + Q = Q + P$ per tutti $P, Q \in E(\mathbb{F}_p)$, rendendo il gruppo **abeliano**. Come in ogni gruppo, per un punto $P \in E(\mathbb{F}_p)$ si scrive $2P := P + P$, $3P := P + P + P$ e, più in generale, $\alpha P := (\alpha - 1)P + P$ per ogni intero positivo α. Si noti che αP può essere calcolato con al più $2 \log_2 \alpha$ operazioni di gruppo usando l'algoritmo di raddoppio e somma (double-and-add).

① SE $P = (x_1, y_1)$ È UN PUNTO $\Rightarrow$ IL SUO INVERSO È $-P = (x_1, -y_1)$ FONDAMENTALE

Esempio. Consideriamo i seguenti due punti $P = (0, 1) \in E = (\mathbb{Z}_5)$ e $Q = (0, 1) \in E = (\mathbb{Z}_5)$, dove $E_5(x, y) : y^2 = x^3 + x + 1$

Osserviamo che $P = Q$ quindi ci troviamo nel secondo caso, di conseguenza:

$$\begin{aligned} \lambda &= (3x_1^2 + a)(2y_1)^{-1} \pmod 5 = 1 \cdot 2^{-1} \pmod 5 = 1 \cdot 3 = 3 \\ x_3 &= \lambda^2 - 2x_1 \pmod 5 = 3^2 \pmod 5 = 4 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \pmod 5 = 3(0 - 4) - 1 \pmod 5 = 3 - 1 = 2 \end{aligned}$$

$P + Q = 2P = (4, 2)$

Esempio. Consideriamo i seguenti due punti $P = (4, 2) \in E = (\mathbb{Z}_5)$ e $Q = (0, 1) \in E = (\mathbb{Z}_5)$, dove $E_5(x, y) : y^2 = x^3 + x + 1$

Osserviamo che $P \neq Q$ quindi ci troviamo nel primo caso, di conseguenza:

$$\begin{aligned} \lambda &= (y_2 - y_1)(x_2 - x_1)^{-1} \pmod 5 = -1 \cdot -4^{-1} \pmod 5 = 4 \cdot 1 = 4 \\ x_3 &= \lambda^2 - x_1 - x_2 \pmod 5 = 4^2 - 4 \pmod 5 = 12 \pmod 5 = 2 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \pmod 5 = 4(2) - 2 \pmod 5 = 1 \end{aligned}$$

$P + Q = (2, 1)$

8.1 EC Group for Cryptography

Si consideri una curva ellittica $E/\mathbb{F}_p$ e il gruppo $E(\mathbb{F}_p)$ dei punti definiti su $\mathbb{F}_p$. È noto che $E(\mathbb{F}_p)$ è un gruppo abeliano finito; supponiamo inoltre che sia ciclico, ossia generato da un punto $P \in E(\mathbb{F}_p)$. Possiamo allora analizzare la difficoltà di problemi come il logaritmo discreto, il Diffie-Hellman computazionale (CDH) e il Diffie-Hellman decisionale (DDH) nel gruppo $E(\mathbb{F}_p)$. Quando tali problemi risultano difficili in $E(\mathbb{F}_p)$, tutte le costruzioni crittografiche viste in precedenza possono essere instanziate in questo gruppo, ottenendo sistemi su curve ellittiche.

Definizione (ECDLP). Sia P un punto di ordine q (cioè $qP = \mathcal{O}$). Dati P e q, insieme a un punto $Q := \alpha P$ per qualche $\alpha \in \mathbb{Z}_q$, si chiede di determinare α. Qui αP indica il punto ottenuto sommando P a sé stesso α volte secondo la legge di addizione del gruppo.

La scelta della curva è cruciale sia per la sicurezza sia per l'efficienza. È preferibile adottare curve raccomandate dagli standard (NIST/SECG, CFRG) e ricordare che non si lavora solo su campi primi $\mathbb{F}_p$, ma anche su campi binari $\mathbb{F}_{2^m}$. Le curve moderne, come le Edwards e le Montgomery, impiegano forme diverse dalla Weierstrass e tendono a essere più efficienti e robuste. Ogni famiglia di curve presenta proprietà e assunzioni differenti (ordine del gruppo, cofattore, *pairing-friendliness*, ipotesi DL/CDH/DDH), con effetti concreti sui protocolli che vi si basano. In particolare, nelle famiglie con mappe bilineari il problema decisionale di Diffie-Hellman (DDH) può risultare facile mentre quello computazionale (CDH) resta difficile; per questo la selezione della curva va allineata alle garanzie richieste dal protocollo.

Consideriamo la moltiplicazione per scalare (analoga all'esponenziazione modulare). In un gruppo di ordine di circa 160 bit, cioè $q \approx 2^{160}$, il costo computazionale è $O(\log q)$: con l'algoritmo *double-and-add* (analogo di *square-and-multiply*) servono in media $\approx \frac{3}{2} \log_2 q$ operazioni di gruppo (raddoppi e somme).

8.2 Algoritmio Double-and-Add

Dato un punto P su una curva ellittica e un intero $k \geq 0$, la moltiplicazione scalare kP si ottiene combinando ripetutamente le operazioni di raddoppio ($2Q$) e somma ($Q + R$). Di seguito i due schemi pratici più usati.

DATI P E K ≥ 0, CALCOLARE Q = kP È FACILE (ALGO DOUBLE-AND-ADD)
DATI Q E P È IMPOSSIBILE RISALIRE A K

Double-and-Add (da sinistra a destra)

Input: $k = (b_{\ell-1} \dots b_1 b_0)_2$, P; Output: $Q = kP$.
1. $Q \leftarrow \mathcal{O}$
2. Per $i = \ell - 1, \dots, 0$:
   (a) $Q \leftarrow 2Q$
   (b) Se $b_i = 1$ allora $Q \leftarrow Q + P$
3. Restituisci Q

Costo: ℓ raddoppi e $\mathrm{wt}(k)$ somme, dove $\mathrm{wt}(k)$ è il numero di bit a 1. In media, per k casuale, si hanno $\approx \ell$ raddoppi e $\approx \ell/2$ somme.

Esempio. Supponiamo di voler calcolare $[1437]P$. Innanzitutto esprimiamo in bits 1437.

$$(1437)_{10} = (10110011101)_2$$

La regola di lettura standard è da $LSB \to MSB$. Inizializziamo $R = \mathcal{O}$

LSB → MSB	Double	Result
1	$1P$	$R = R + P$
0	$2P$	-
1	$4P$	$R = R + 4P = 5P$
1	$8P$	$R = R + 8P = 13P$
1	$16P$	$R = R + 16P = 29P$
0	$32P$	-
0	$64P$	-
1	$128P$	$R = R + 128P = 157P$
1	$256P$	$R = R + 256P = 413P$
0	$512P$	-
1	$1024P$	$R = R + 1024P = 1437P$

Sicurezza La sicurezza delle operazioni su curve ellittiche si basa su problemi considerati difficili:

- Dato un punto generatore P e un punto $Q = [k]P$, risalire allo scalare k è computazionalmente difficile. Questo è il **Problema del Logaritmo Discreto su Curve Ellittiche (ECDLP)**.
- In molte curve si assumono difficili anche:

  Computational Diffie-Hellman (CDH): dati P, $[a]P$, $[b]P$, calcolare $[ab]P$;

  Decisional Diffie-Hellman (DDH): distinguere

  $$(P,\ [a]P,\ [b]P,\ [ab]P) \text{ da } (P,\ [a]P,\ [b]P,\ [c]P)$$

  con c casuale.

8.3 ECDH

L'obiettivo di ECDH è permettere a due parti (Alex A e Bob B) di stabilire un segreto condiviso su un canale pubblico, senza rivelare le chiavi private.

Parametri comuni (setup) Si fissano i parametri di dominio

$$\mathsf{DP} = (p, a, b, G, q),$$

dove:

- $E/\mathbb{F}_p$: $y^2 \equiv x^3 + ax + b \pmod p$ con discriminante non nullo;
- $G \in E(\mathbb{F}_p)$ è un generatore di ordine primo q;

EC DH IDENTICO A DH, SEMPLICEMENTE COME GRUPPO SI SCEGLIE UN GRUPPO EC

Protocollo

1. Generazione delle chiavi:

$$x \leftarrow_{u.a.r.} \mathbb{Z}_q^*, \quad X = [x]G \qquad \text{e} \qquad y \leftarrow_{u.a.r.} \mathbb{Z}_q^*, \quad Y = [y]G.$$

A & invia X a B, B invia Y a A.

2. Calcolo del segreto condiviso:

$$S_A = [x]Y = [x]([y]G) = [xy]G, \qquad S_B = [y]X = [y]([x]G) = [xy]G.$$

Entrambe le parti ottengono lo stesso punto $S = [xy]G$.

3. Derivazione della chiave:

$$k = \mathrm{KDF}\big(\mathrm{enc}(S)\big) \quad \text{(es. usando la coordinata } x \text{ di } S \text{ e un salt).}$$

Note operative

- Validare le chiavi pubbliche ricevute: $P \neq \mathcal{O}$, coordinate in $\mathbb{F}_p$, P sulla curva e nel sottogruppo di ordine q (ad es. verificando $[q]P = \mathcal{O}$).
- Usare chiavi effimere (ECDHE) per la Perfect Forward Secrecy: generare coppie fresh per sessione e autenticare le chiavi pubbliche per evitare MITM.
- Non usare direttamente il punto S come chiave: derivare sempre le chiavi con una KDF (es. HKDF) con salt/label e contesto (transcript, identità).

8.4 ECDSA

L'ECDSA (Elliptic Curve Digital Signature Algorithm) fornisce autenticità e integrità su gruppi di punti ellittici ed è basato sulla difficoltà dell'ECDLP.

Parametri e chiavi

- Parametri di dominio $\mathsf{DP} = (p, a, b, G, q)$ su $E/\mathbb{F}_p$, con $G \in E(\mathbb{F}_p)$ generatore di ordine primo q.
- Funzione di hash $H : \{0,1\}^* \to \{0,1\}^*$ e digest ridotto $(H(m))$.
- Chiave privata: $d \in \{1, \ldots, q-1\}$.
- Chiave pubblica: $Q = [d]G$.

Firma di un messaggio m

1. Calcola $(H(m))$.
2. Scegli $k \leftarrow_{u.a.r.} \{1, \ldots, q-1\}$ (nonce unico e segreto).
3. Poni $R = [k]G = (x_1, y_1)$ e $r = x_1 \bmod q$; se $r = 0$ ripeti dal punto 2.
4. Calcola $s = k^{-1}(H(m) + d\,r) \bmod q$; se $s = 0$ ripeti dal punto 2.
5. La firma è la coppia (r, s).

Verifica di (r, s) su m con Q

1. Rifiuta se $r \notin \{1, \ldots, q-1\}$ o $s \notin \{1, \ldots, q-1\}$.
2. Calcola $H(m)$ e $w = s^{-1} \bmod q$.

3. Poni $u_1 = H(m)\, w \bmod q$, $u_2 = r\, w \bmod q$.
4. Calcola $X = [u_1]G + [u_2]Q = (x_0, y_0)$; se $X = \mathcal{O}$ rifiuta.

$$\begin{aligned} X &= [u_1]G + [u_2]Q \\ &= [H(m)w]G + [rw]Q \\ &= [H(m)s^{-1}]G + [rs^{-1}]Q \\ &= [H(m)s^{-1}]G + [rs^{-1}d]G \\ &= [(H(m) + rd)s^{-1}]G \\ &= [k]G = R, \end{aligned}$$

dove $s \equiv k^{-1}(H(m) + rd) \pmod q$ implica $(H(m) + rd)s^{-1} \equiv k \pmod q$.

5. Accetta se $x_0 \bmod q = r$, altrimenti rifiuta.

Note operative

- Nonce k: deve essere unico e imprevedibile; è consigliato utilizzare RFC 6979 per un nonce deterministico. **Non deve essere mai divulgato!**
- Validazione di Q: verifica che $Q \neq \mathcal{O}$, che $Q \in E(\mathbb{F}_p)$ e che $[q]Q = \mathcal{O}$.

Cosa accade se k è prevedibile? In ECDSA la sicurezza richiede che k sia un nonce segreto, unico e imprevedibile per ogni firma. Se un avversario conosce k, da (r, s) e $H(m)$ ricava subito la chiave privata:

$$s \equiv k^{-1}(H(m) + r\, d) \;\Rightarrow\; sk \equiv H(m) + r\, d \;\Rightarrow\; d \equiv (sk - H(m))\, r^{-1} \pmod q.$$

Cosa accade se k viene riutilizzato? Firmando due messaggi diversi m_1, m_2 con lo stesso k si ottiene lo stesso r e si può recuperare k (e quindi d) anche senza conoscerlo a priori. Ponendo $z_i = H(m_i)$:

$$\begin{aligned} s_1 &\equiv k^{-1}(z_1 + r\, d) \pmod q, \\ s_2 &\equiv k^{-1}(z_2 + r\, d) \pmod q, \end{aligned}$$

sottraendo:

$$s_1 - s_2 \equiv k^{-1}(z_1 - z_2) \pmod q \;\Rightarrow\; k \equiv (z_1 - z_2)\,(s_1 - s_2)^{-1} \pmod q.$$

Infine:

$$d \equiv (s_1\, k - z_1)\, r^{-1} \pmod q.$$

Quindi il riuso di k compromette la chiave privata.

9 Bilinear Maps

Una mappa bilineare (o pairing), indicata di solito con e, è una funzione che collega due gruppi crittografici G_1 e G_2 producendo un elemento in un terzo gruppo G_T.

Definizione (Mappa bilineare). Siano G_1, G_2, G_T gruppi ciclici di ordine primo q, con generatori $g_1 \in G_1$ e $g_2 \in G_2$. Una funzione

$$e : G_1 \times G_2 \to G_T$$

è detta una mappa bilineare se soddisfa:

SE VALE 1) => e = MAPPA BILINEARE
SE VALGONO 2) e 3) => e = MAPPA BILINEARE AMMISSIBILE

1. **Bilinearità:** per ogni $u \in G_1$, $v \in G_2$ e $a, b \in \mathbb{Z}_q$,
$$e(u^a, v^b) = e(u, v)^{ab}.$$
2. **Non-degenerazione:** $e(g_1, g_2) \neq 1_{G_T}$ (equivalentemente, $e(g_1, g_2)$ genera G_T).
3. **Computabilità efficiente:** e è calcolabile in tempo polinomiale.

Nota Se usiamo la notazione additiva tipica delle curve ellittiche, la formula diventa $e([a]P, [b]Q) = e(P, Q)^{ab}$

Dalla bilinearità seguono le identità:

$$e(u_1 u_2, v) = e(u_1, v) \cdot e(u_2, v), \qquad e(u, v_1 v_2) = e(u, v_1) \cdot e(u, v_2)$$

Esempio. Semplificazione dell'espressione utilizzando le proprietà delle mappe bilineari:

$$\begin{aligned}
e(ag^b, g^{c+d}) &= e(g^{ab}, g^{c+d}) && \text{(Moltiplicazione scalare come esponente: } ag^b \to g^{ab}) \\
&= e(g^{ab}, g^c \cdot g^d) \\
&\overset{\text{linearità}}{=} e(g^{ab}, g^c) \cdot e(g^{ab}, g^d) && \text{(Distribuzione sul secondo argomento)} \\
&\overset{\text{bilinearità}}{=} e(g, g)^{abc} \cdot e(g, g)^{abd} && \text{(Estrazione degli esponenti in } G_t) \\
&= e(g, g)^{abc+abd} \\
&= g_t^{ab(c+d)}
\end{aligned}$$

Nel caso $G_1 = G_2$ si parla di pairing simmetrico; altrimenti è asimmetrico.

Da ora in poi supponiamo sempre che $G = G_1 = G_2$.

In pratica, il gruppo G è scelto quasi sempre come l'insieme dei punti di una curva ellittica (o di un suo sottogruppo) definita su un campo finito. Un esempio elementare è la curva $y^2 = x^3 + 1$ su $\mathbb{F}_p$, che fornisce un gruppo abeliano efficiente su cui definire la mappa bilineare.

Più in generale, G può essere visto come una varietà abeliana su un campo: le curve ellittiche ne sono il caso di dimensione 1, ma sono state studiate anche varietà abeliane di dimensione superiore per ottenere pairing con proprietà specifiche.

Il gruppo di arrivo G_T è tipicamente il gruppo moltiplicativo di un campo finito; è qui che vive l'output del pairing e dove si sfruttano le identità indotte dalla bilinearità.

9.1 Decisional Diffie-Hellman

Un primo problema affrontabile con le mappe bilineari è il *Decisional Diffie-Hellman (DDH)*.

Definizione (DDH). Sia G un gruppo ciclico di ordine primo q con generatore g. Il vantaggio di un algoritmo probabilistico $\mathcal{A}$ nel distinguere una tupla Diffie–Hellman valida da una casuale è

$$\mathrm{Adv}^{\mathrm{DDH}}_{\mathcal{A},G} = \left|\Pr\left[\mathcal{A}(g, g^a, g^b, g^{ab}) = 1\right] - \Pr\left[\mathcal{A}(g, g^a, g^b, g^z) = 1\right]\right|,$$

dove $a, b, z \leftarrow_{u.a.r.} \mathbb{Z}_q$.

Se l'Advantage è **vicino a 0**, l'algoritmo sta praticamente tirando a indovinare; quindi il DDH è difficile (buono per la sicurezza classica).

Se l'Advantage è **vicino a 1**, l'algoritmo distingue perfettamente i due casi; quindi il DDH è **facile**.

In un gruppo crittografico standard, distinguere una quaterna di Diffie-Hellman (g, g^a, g^b, g^{ab}) da una quaterna casuale (g, g^a, g^b, g^z) è ritenuto impossibile per un computer in tempi ragionevoli. Tuttavia, se disponiamo di una mappa bilineare $e : G \times G \to G_t$, un algoritmo può risolvere questo problema in tempo polinomiale.

Con una mappa bilineare $e : G \times G \to G_T$, DDH diventa facile: dato (g, g^a, g^b, g^z), si verifica

$$e(g^a, g^b) \stackrel{?}{=} e(g, g^z).$$

Grazie alla proprietà di bilinearità, sappiamo che $e(g^a, g^b) = e(g, g)^{ab}$. Se $z = ab$, allora anche $e(g, g^z) = e(g, g)^{ab}$. Se i due risultati coincidono, la quaterna è valida.

Esempio. Immagina di ricevere quattro valori e di voler verificare se formano una quaterna di Diffie–Hellman valida $(P, [2]P, [3]P, [6]P)$ con $[6]P = [2 \cdot 3]P$.

Test con la mappa bilineare. Poni $Z = e(P, P) \in G_T$. Allora:

$$e([2]P, [3]P) = e(P, P)^{2\cdot 3} = Z^6, \qquad e(P, [6]P) = e(P, P)^6 = Z^6.$$

Poiché i due risultati coincidono, la quaterna è valida.

Se invece l'ultimo valore fosse $[7]P$, avremmo

$$e(P, [7]P) = e(P, P)^7 = Z^7 \neq Z^6,$$

e la quaterna non sarebbe valida.

Nota su Computationl Diffie-Hellman (CDH)

È utile sottolineare che il problema computazionale di Diffie-Hellman (CDH) può rimanere difficile in un gruppo ciclico G anche quando il problema decisionale (DDH) è facile, ad esempio grazie alla disponibilità di una mappa bilineare. Non è infatti noto, in generale, come usare un pairing per risolvere CDH. Si definisce *gap Diffie-Hellman* (GDH) un gruppo di ordine primo in cui DDH è facile ma CDH resta difficile; questa definizione non dipende dall'esistenza di una mappa bilineare. Le mappe bilineari si possono quindi interpretare come uno strumento per ottenere (o modellare) gruppi con tale "gap": rendono semplice distinguere le tuple di Diffie-Hellman valide (DDH), senza offrire un metodo efficiente per calcolare $[ab]P$ a partire da $[a]P$ e $[b]P$ (CDH).

Logaritmo Discreto

Teorema 1. *Se esiste una mappa bilineare $e : G \times G \to G_T$, allora il problema del logaritmo discreto in G non è più difficile del problema del logaritmo discreto in G_T (riduzione in tempo polinomiale).*

Dati $g \in G$ e $g^a \in G$, si calcolano $e(g, g) \in G_T$ ed $e(g, g^a) = e(g, g)^a \in G_T$. A questo punto si applica un risolutore del logaritmo discreto in G_T per ricavare a. Questa procedura è nota come riduzione di Menezes–Okamoto–Vanstone (MOV).

9.2 Joux's 3-Party Diffie-Hellman

Sia G un gruppo ciclico di ordine primo q, $e : G \times G \to G_T$ una mappa bilineare, e g un generatore di G. Poni $\hat{g} = e(g, g) \in G_T$.

Protocollo.

1. Alice, Bob e Carol scelgono $a, b, c \leftarrow_{u.a.r.} \mathbb{Z}_q$.
2. Trasmettono pubblicamente g^a, g^b, g^c.
3. Calcolo del segreto condiviso (uguale per tutti grazie alla bilinearità):

$$\text{Alice: } e(g^b, g^c)^a = \hat{g}^{abc}, \quad \text{Bob: } e(g^c, g^a)^b = \hat{g}^{abc}, \quad \text{Carol: } e(g^a, g^b)^c = \hat{g}^{abc}.$$

La chiave finale si deriva come $k = \mathrm{KDF}\big(\mathrm{enc}(\hat{g}^{abc})\big)$.

di "BARARE"

Dal punto di vista di Alice, la mappa bilineare consente un "trucco" per ottenere $\hat{g}^{bc}$ a partire da g^b e g^c:

$$e(g^b, g^c) = e(g, g)^{bc} = \hat{g}^{bc}.$$

Poi una normale esponenziazione porta a $\hat{g}^{abc}$:

$$(\hat{g}^{bc})^a = \hat{g}^{abc}.$$

Non è possibile usare e in modo annidato per ottenere direttamente $\hat{g}^{abc}$ da g^a, g^b, g^c, perché il secondo argomento di e deve appartenere a G:

$$e\big(g^a,\, e(g^b, g^c)\big) = e(g^a, \hat{g}^{bc}) \neq \hat{g}^{abc} \quad \text{poiché } \hat{g}^{bc} \notin G$$

In sintesi, è consentita una **sola applicazione** del pairing.

9.3 Boneh and Franklin's IBE Scheme

Definizione (Identity-Based Encryption (IBE) di Boneh.Franklin). L'idea è cifrare direttamente verso un'identità testuale (es. un'email) usata come chiave pubblica, eliminando certificati e gestione PKI. Un'autorità fidata (PKG - Private Key Generator) detiene una *master key* e, su richiesta, estrae la chiave privata associata all'identità. Lo schema di Boneh-Franklin realizza IBE usando mappe bilineari: la sicurezza si basa sulla difficoltà di Diffie-Hellman bilineare (BDH) e sull-uso di funzioni di hash in modello random oracle.

Sia G un gruppo ciclico di ordine primo q, $e : G \times G \to G_T$ una mappa bilineare, e g un generatore di G. Poni $\hat{g} = e(g, g) \in G_T$. Siano $h_1 : \{0,1\}^* \to G$ e $h_2 : G_T \to \{0,1\}^*$ funzioni di hash. Tutti questi sono parametri pubblici.

MAPPA STRINGA IN $x \in G$ — MAPPA $x \in G$ IN STRINGA

SETUP: La PKG estrare $s \leftarrow_{u.a.r.} \in \mathbb{Z}_q$ e calcola la chiave pubblica g^s.

CIFRATURA: Alice vuole inviare un messaggio m a Bob. Estrae $r \leftarrow_{u.a.r.} \in \mathbb{Z}_q$ e calcola:

$$\begin{aligned} ENC(g,\ g^s,\ \text{"Bob"},\ m) &= (g^r, m \oplus h_2(e(h_1(\text{"Bob"}),\ g^s)^r)) \\ &= (g^r, m \oplus h_2(e(h_1(\text{"Bob"}),\ g)^{sr})) \end{aligned}$$

GENERAZIONE PK: Ora, la PKG crea la chiave privata di Bob nel seguente modo:

$$\texttt{MakeKey}(s, \text{"Bob"}) = h_1(\text{"Bob"})^s$$

DECIFRATURA: Ricevuto il messaggio cifrato $(u, v) = (g^r, m \oplus h_2(e(h_1(\text{"Bob"}),\ g)^{sr}))$, usa la chiave privata $w = h_1(\text{"Bob"})^s$, Bob decifra nel seguente modo:

$$\begin{aligned} DEC(u, v, w) &= v \oplus h_2(e(w, u)) \\ &= m \oplus h_2(e(h_1(\text{"Bob"}),\ g)^{sr}) \oplus h_2(e(h_1(\text{"Bob"})^s,\ g^r)) \\ &= m \oplus h_2(e(h_1(\text{"Bob"}),\ g)^{sr}) \oplus h_2(e(h_1(\text{"Bob"}),\ g)^{sr}) \\ &= m \end{aligned}$$

SCHEMA BONEH-FRANKLIN

Pensiamo a tre attori che cooperano per derivare una chiave condivisa. Alice (mittente) sceglie r e pubblica $u = g^r$. La PKG (autorità) possiede il segreto s e pubblica g^s. Bob (destinatario) ha come "chiave pubblica" $h_1(\text{"Bob"}) \in G$, che si può immaginare come $h_1(\text{"Bob"}) = g^t$ con t ignoto. L'obiettivo è ottenere $\hat{g}^{rst}$, dove $\hat{g} = e(g, g)$, da usare come chiave di sessione.

Come Bob ottiene la chiave. Bob non conosce t, quindi la PKG gli fornisce in modo sicuro la chiave privata $w = h_1(\text{"Bob"})^s = g^{st}$. Avendo $u = g^r$ dal cifrato di Alice, Bob calcola

$$e(w, u) = e(g^{st}, g^r) = e(g, g)^{rst} = \hat{g}^{rst},$$

e deriva la chiave con $h_2(e(w, u))$ per decifrare.

È consentita una sola applicazione del pairing: ricevendo un punto in G ($w = g^{st}$) Bob può combinare w e u via $e(\cdot, \cdot)$. Se invece ricevesse direttamente un elemento in G_T (es. $\hat{g}^{st}$), non potrebbe più comporlo con u.

